CSC314 - OPERATION SYSTEM

2 OBJECTIVES AND OUTCOME

Objectives

- To introduce operating systems concepts
- Show how an operating system may manage computer hardware (processor, memory, I/O devices).

Outcome

- An understanding of an operating system and how it abstracts the complexity of the computer hardware by presenting the user with a simple interface that is easy to understand and use.

COURSE OUTLINE

Introduction:

- Definitions, goals, computer system components, Abstract view of systems components

Functions of operating systems

- Process management, Memory Management, File Management, I/O system Management, secondary storage management,

Evolution of Operating System

- Main frame OS, batch processing OS, Multiprogramming OS, Time sharing (multitasking) OS, Multiprocessor OS, Distributed OS, …

COURSE OUTLINE

OS Services:

- Program execution, I/O operations, File systems manipulation, communication, error detection, resource allocation, accounting, protection, System call, ….

Computer System Architecture

- Computer System Operations, Interruption, I/O interrupt structure, synchronous versus asynchronous methods, Direct memory access method, storage structure, storage hierarchy,

Process concepts and management

- Definition, states, process control bloc, Process scheduling, scheduler, context switch, process life cycle, ….

COURSE OUTLINE

CPU scheduling:

- Basic Concepts, Scheduling Criteria, Scheduling Algorithms, Multiple-Processor Scheduling, Real-Time Scheduling, Algorithm Evaluation, …

Threads

- Single-threaded versus multithreaded, resource sharing,

Inter-process communication

- Mechanism for processes to communicate and to synchronize their actions, send(message) and receive(message), communication link, concurrent process, …

COURSE OUTLINE

Process synchronization

- Producer–consumer problem, critical section and proposed solutions,
- Readers-writers problem,
- Dining phylosophers problem
- Deadlocks

Memory management

- Definitions, types of allocations, Virtual memory

File systems interface

- Concept, attributes, operations, access methods, …
- Directory structure

COURSE OUTLINE

Secondary storage structure

- Disks structure, disk scheduling, disk management
- RAID structure,….

I/O System

- Definitions, concepts, application interface, kernel I/O subsystem, from I/O request to hardware operations

INTRODUCTION

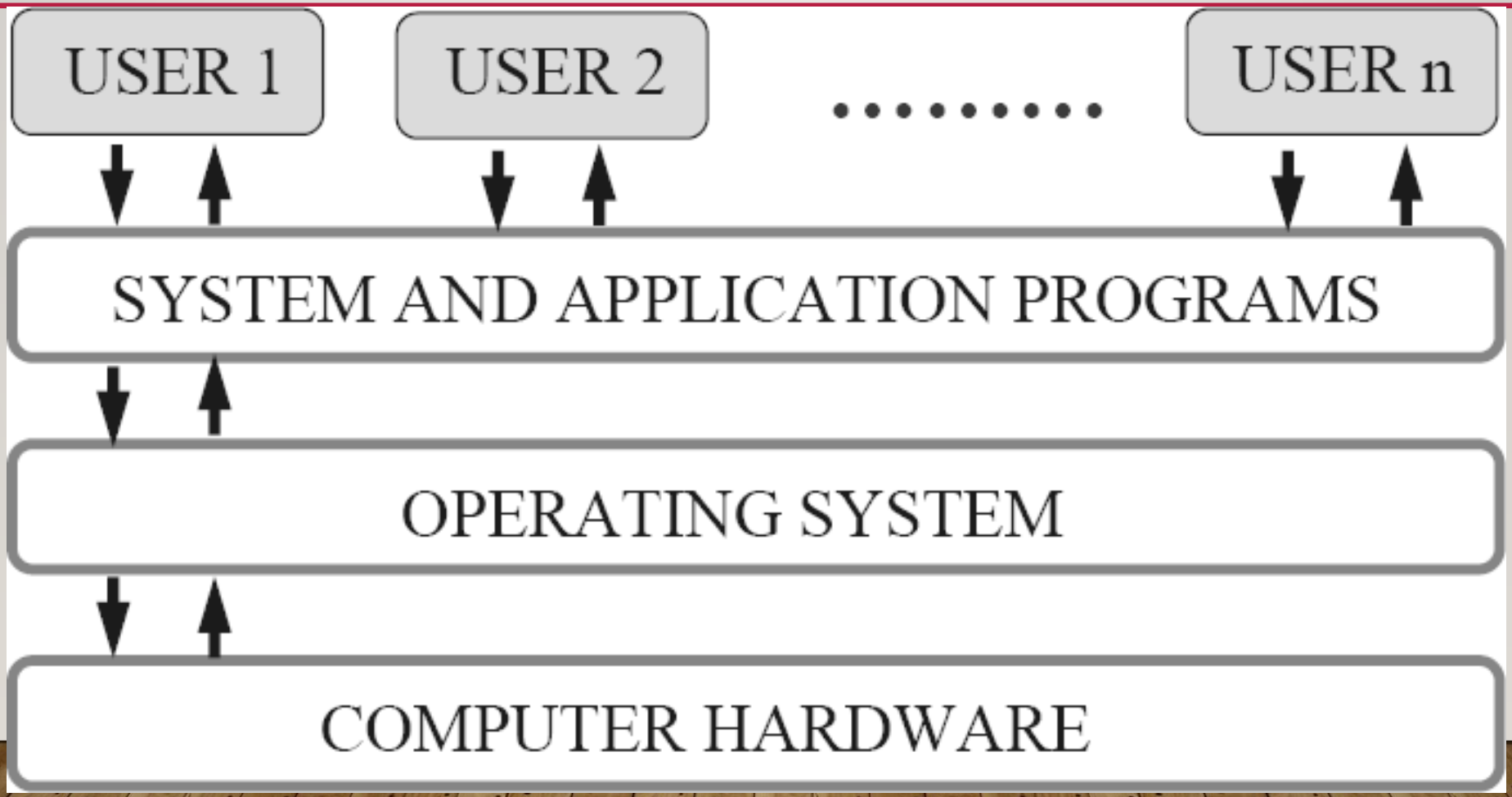
9 INTRODUCTION

- What is an Operating System?
  - A program that acts as an intermediary between a user of a computer and the computer hardware.
  - A collection of system programs that together control the operations of a computer system.
- Examples: UNIX, MS-DOS, MS-Windows, Windows/NT, OS/2, MacOS, Android, …
- OS goals:
  - Execute user programs and make solving user problems easier.
  - Make the computer system convenient to use.
  - Use the computer hardware in an efficient manner.

10 INTRODUCTION

- Computer System Components
  1. Hardware – provides basic computing resources (CPU, memory, I/O devices).
  2. Operating system – controls and coordinates the use of the hardware among the various application programs for the various users.
  3. Applications programs – Define the ways in which the system resources are used to solve the computing problems of the users (compilers, database systems, video games, business programs).
  4. Users (people, machines, other computers).

11 INTRODUCTION



12 INTRODUCTION

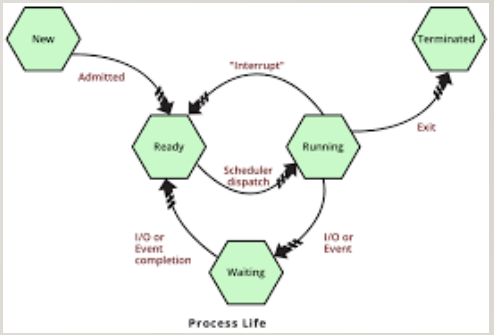
- Some OS Definitions
  - Resource allocator – manages and allocates resources.
  - Control program – controls the execution of user programs and operations of I/O devices .
  - Kernel – The one program running at all times (all else being application programs).
- Components of OS: OS has two parts.
  1. Kernel: it is an active part of an OS i.e., the part of OS running at all times. It is a programs which can interact with the hardware (Device driver, dll files, system files etc.)
  2. Shell: called as the command interpreter. It is a set of programs used to interact with the application programs. It is responsible for execution of instructions given to OS.

13 INTRODUCTION

- Operating systems can be explored from two viewpoints
  - User View: From the user's point view, the OS is designed for one user to monopolize its resources, to maximize the work that the user is performing and for ease of use.
  - System View: From the computer's point of view, an operating system is a control program that manages the execution of user programs to prevent errors and improper use of the computer. It is concerned with the operation and control of I/O devices.

14 INTRODUCTION

- Functions of an OS: Process Management
  - A process is a program in execution. A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task.
  - The operating system is responsible for the following activities in connection with process management.
    - Process creation and deletion.
    - Process suspension and resumption.
    - Provision of mechanisms for: process synchronization and process communication.



15 INTRODUCTION

- Main-Memory Management
  - Memory (main) is a large array of words or bytes, each with its own address. It is a repository of quickly accessible data shared by the CPU and I/O devices.
  - Main memory is a volatile storage device. It loses its contents in the case of system failure.
  - The operating system is responsible for the following activities in connections with memory management:
    - Keep track of which parts of memory are currently being used and by whom.
    - Decide which processes to load when memory space becomes available.
    - Allocate and de-allocate memory space as needed.

16 INTRODUCTION

- File Management
  - A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data.
  - The operating system is responsible for the following activities in connections with file management:
    - File creation and deletion, directory creation and deletion.
    - Support of primitives for manipulating files and directories.
    - Mapping files onto secondary storage.
    - File backup on stable (nonvolatile) storage media.

17 INTRODUCTION

- I/O System Management
  - The I/O system consists of:
    - A buffer-caching system
    - A general device-driver interface
    - Drivers for specific hardware devices

18 INTRODUCTION

- Secondary-Storage Management
  - Main memory is volatile and too small to accommodate all data and programs permanently. Computer system must provide secondary storage to back up main memory.
    - Most modern computer systems use disks as the principle on-line storage medium, for both programs and data.
  - The operating system is responsible for the following activities in connection with disk management:
    - Free space management
    - Storage allocation
    - Disk scheduling

19 INTRODUCTION

- Networking (Distributed Systems)
  - A distributed system is a collection processors that do not share memory or a clock. Each processor has its own local memory.
  - The processors in the system are connected through a communication network.
  - Communication takes place using a protocol.
  - A distributed system provides user access to various system resources.

20 INTRODUCTION

- Networking (Distributed Systems)
  - Access to a shared resource allows:
    - Computation speed-up
    - Increased data availability
    - Enhanced reliability

21 INTRODUCTION

- Protection System
  - Protection refers to a mechanism for controlling access by programs, processes, or users to both system and user resources.
  - The protection mechanism must:
    - Distinguish between authorized and unauthorized usage.
    - Specify the controls to be imposed.
    - Provide a means of enforcement.

22 INTRODUCTION

- Command-Interpreter System
  - Many commands are given to the operating system by control statements which deal with:
    - Process creation and management, I/O handling
    - Secondary-storage management, Main-memory management
    - File-system access, Protection
    - Networking

23 INTRODUCTION

- Operating-System Structures
  - System Components
  - Operating System Services
  - System Calls
  - System Programs
  - System Structure
  - Virtual Machines
  - System Design and Implementation
  - System Generation
- Common System Components
  - Process Management, Main Memory Management
  - File Management, I/O System Management
  - Secondary Memory Management
  - Networking, Protection System
  - Command-Interpreter System

EVOLUTION OF OPERATING SYSTEMS (OS)

25 EVOLUTION OF OS

Mainframe Systems

- Reduce setup time by batching similar jobs Automatic job sequencing – automatically transfers control from one job to another. First rudimentary operating system. Resident monitor.
- Initial control in monitor
- Control transfers to job
- When job completes control transfers pack to monitor

26 EVOLUTION OF OS

Batch Processing System:

- This type of OS accepts more than one job and these jobs are batched/ grouped together according to their similar requirements. This is done by computer operator. Whenever the computer becomes available, the batched jobs are sent for execution and gradually the output is sent back to the user.
- It allowed only one program at a time.
- This OS is responsible for scheduling the jobs according to priority and the resource required.

27 EVOLUTION OF OS

Batch Processing System: Characteristics

- No Interaction
- Batching Jobs
- Sequential Execution
- Automatic Transition

28 EVOLUTION OF OS

Multiprogramming Operating System:

- Used to execute more than one jobs simultaneously by a single processor. it increases CPU utilization by organizing jobs so that the CPU always has one job to execute.
- Multiprogramming operating systems use the mechanism of job scheduling and CPU scheduling.
- The concept of multiprogramming is described as follows:
  - All the jobs that enter the system are stored in the job pool. The operating system loads a set of jobs from job pool into main memory and begins to execute.

29 EVOLUTION OF OS

Multiprogramming Operating System:

- The concept of multiprogramming is described as follows:
  - During execution, the job may have to wait for some task, such as an I/O operation, to complete. In a multiprogramming system, the operating system simply switches to another job and executes. When that job needs to wait, the CPU is switched to another job, and so on.
  - When the first job finishes waiting and it gets the CPU back.
  - As long as at least one job needs to execute, the CPU is never idle.

30 EVOLUTION OF OS

Time-Sharing Operating Systems

- Time sharing (or multitasking) OS is a logical extension of multiprogramming. It provides extra facilities such as:
  - Faster switching between multiple jobs to make processing faster.
  - Allows multiple users to share computer system simultaneously.
  - The users can interact with each job while it is running.

31 EVOLUTION OF OS

Time-Sharing Operating Systems

- These systems use a concept of virtual memory for effective utilization of memory space.
- In this OS, no jobs are discarded. Each one is executed using virtual memory concept.
- It uses CPU scheduling, memory management, disc management and security management.

32 EVOLUTION OF OS

Multiprocessor Operating Systems

- Known as parallel OS or tightly coupled OS.
- They have more than one processor in close communication that sharing the computer bus, the clock and sometimes memory and peripheral devices.
- It executes multiple jobs at same time and makes the processing faster.

33 EVOLUTION OF OS

Multiprocessor Operating Systems

- Increased throughput: By increasing the number of processors, the system performs more work in less time. The speed-up ratio with N processors is less than N.
- Economy of scale: Multiprocessor systems can save more money than multiple single-processor systems, because they can share peripherals, mass storage, and power supplies.
- Increased reliability: If one processor fails to done its task, then each of the remaining processors must pick up a share of the work of the failed processor.
- The failure of one processor will not halt the system, only slow it down.
- The ability to continue providing service proportional to the level of surviving make the system **fault tolerant**.

34 EVOLUTION OF OS

Multiprocessor Operating Systems

- Two categories: Symmetric multiprocessing and Asymmetric multiprocessing system
  - In **symmetric** multiprocessing system, each processor runs an identical copy of the operating system, and these copies communicate with one another as needed.
  - In **asymmetric** multiprocessing system, a processor is called master processor that controls other processors called slave processor. Thus, it establishes master-slave relationship. The master processor schedules the jobs and manages the memory for entire system.

35 EVOLUTION OF OS

Distributed Operating Systems

- Different machines are connected in a network, and each machine has its own processor and own local memory.
- The operating systems on all the machines work together to manage the collective network resource.
- It can be classified into two categories:
  - Client-Server systems
  - Peer-to-Peer systems

36 EVOLUTION OF OS

Distributed Operating Systems

- Advantages of distributed systems.
- Resources Sharing
- Computation speed up – load sharing
- Reliability, Communications
- Requires networking infrastructure.
- Local area networks (LAN) or Wide area networks (WAN)

37 EVOLUTION OF OS

Real-Time Operating Systems (RTOS)

- RTOS is a multitasking operating system intended for applications with fixed deadlines (real-time computing).
- Such applications include some small embedded systems, automobile engine controllers, industrial robots, spacecraft, industrial control, and some large-scale computing systems.
- Two categories: Hard and soft RTOS

38 EVOLUTION OF OS

Real-Time Operating Systems (RTOS)

- A **hard real-time** system guarantees that critical tasks be completed on time. This goal requires that all delays in the system be bounded, from the retrieval of stored data to the time that it takes the operating system to finish any request made of it. Such time constraints dictate the facilities that are available in hard real-time systems.
- A **soft real-time** system is a less restrictive type of real-time system. Here, a critical real-time task gets priority over other tasks and retains that priority until it completes. Soft real time system can be mixed with other types of systems. Due to less restriction, they are risky to use for industrial control and robotics

39 EVOLUTION OF OS

Desktop (Personal Computer) Systems

- It is designed for maximizing user convenience and responsiveness. This system is neither multi-user nor multitasking.
- It include PCs running Microsoft Windows and the Apple Macintosh.
- The MS-DOS operating system from Microsoft has been superseded by multiple flavors of Microsoft Windows and IBM has upgraded MS-DOS to the OS/2 multitasking system.
- The Apple Macintosh operating system has been ported to more advanced hardware and then includes new features such as virtual memory and multitasking.

40 EVOLUTION OF OS

Mobile Operating Systems

- A mobile operating system (OS) is software designed to run smartphones, tablets, and wearable devices, acting as the interface between hardware and applications.
- It manages memory, processes, power, and connectivity.
- Key features include application stores, GPS, email, and multitasking.
- The dominant systems are Android (google) and iOS (apple proprietary)

OPERATING SYSTEM (OS) SERVICES

42 OS SERVICES

Five services provided by operating systems (convenience of the users).

1. **Program Execution**: the purpose of computer systems is to allow the user to execute programs. So the operating system provides an environment where the user can conveniently run programs. Running a program involves the allocating and deallocating memory, CPU scheduling in case of multiprocessing.
2. **I/O Operations**: each program requires an input and produces output. This involves the use of I/O. So the operating systems are providing I/O makes it convenient for the users to run programs.
3. **File System Manipulation**: the output of a program may need to be written into new files or input taken from some files. The operating system provides this service.

43 OS SERVICES

Five services provided by operating systems (convenience of the users).

4. **Communications:** the processes need to communicate with each other to exchange information during execution. It may be between processes running on the same computer or running on the different computers. Communications can occur in two ways: shared memory or message passing
5. **Error Detection**: one part of the system may cause malfunctioning of the complete system. To avoid such a situation operating system constantly monitors the system for detecting the errors. This relieves the user of the worry of errors propagating to various part of the system and causing malfunctioning.

44 OS SERVICES

Three services provided by operating systems (for operation of the system itself).

1. **Resource allocation**: when multiple users are logged on the system or multiple jobs are running at the same time, resources must be allocated to each of them. Many different types of resources are managed by the operating system.
2. **Accounting**: the operating systems keep track of which users use how many and which kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics.
3. **Protection**: when several disjointed processes execute concurrently, it should not be possible for one process to interfere with the others, or with the operating system itself. Protection involves ensuring that all access to system resources is controlled. Security of the system from outsiders is also important.

45 OS SERVICES

System Call

- System calls provide an interface between the process and the operating system.
- System calls allow user-level processes to request some services from the operating system which process itself is not allowed to do.
  - Example: for I/O a process involves a system call telling the operating system to read or write a particular area of the memory and this request is satisfied by the operating system (only, and in kernel mode).

46 OS SERVICES

Types of System Call

1. **Process control**
   - end, abort
   - load, execute
   - create process, terminate process
   - get process attributes, set process attributes
   - wait for time
   - wait event, signal event
   - allocate and free memory
2. **File management**
   - create file, delete file
   - open, close
   - read, write, reposition
   - get file attributes, set file attributes

47 OS SERVICES

Types of System Call

3. Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

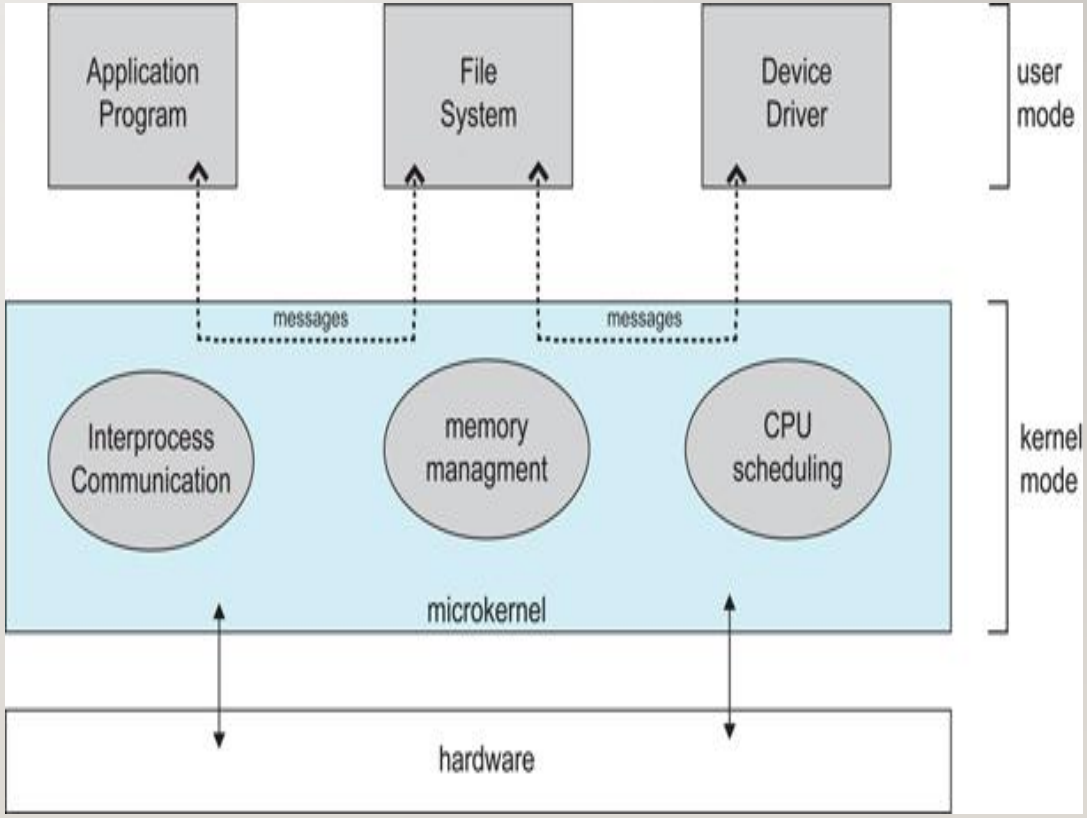
4. Communications

- create, delete communication connection
- send, receive messages
- transfer status information
- attach or detach remote devices

48 OS SERVICES

Types of System Call

5. **Information maintenance**
   - get time or date, set time or date
   - get system data, set system data
   - get process, file, or device attributes
   - set process, file, or device attributes.



49 OS SERVICES

Virtual Machines

- A virtual machine takes the layered approach to its logical conclusion. It treats hardware and the operating system kernel as though they were all hardware.
- A virtual machine provides an interface identical to the underlying bare hardware.
- The operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.
- The resources of the physical computer are shared to create the virtual machines.
  - CPU scheduling can create the appearance that users have their own processor.
  - Spooling and a file system can provide virtual card readers and virtual line printers.
  - A normal user time-sharing terminal serves as the virtual machine operator's console.

50 OS SERVICES

System Generation (SysGen)

- Operating systems are designed to run on any of a class of machines at a variety of sites with a variety of peripheral configurations.
- The system must then be configured or generated for each specific computer site, a process sometimes known as system generation (SYSGEN).
- SYSGEN program obtains information concerning the specific configuration of the hardware system.
- To generate a system, we use a special program. The SYSGEN program reads from a given file, or asks the operator of the system for information concerning the specific configuration of the hardware system, or probes the hardware directly to determine what components are there.

51 OS SERVICES

System Generation (SysGen)

- Information's to be determined by SysGen
- What CPU will be used? What options (extended instruction sets, floating point arithmetic, …) are installed? For multiple-CPU systems, each CPU must be described.
- How much memory is available?
  - Some systems will determine this value themselves by referencing memory location after memory location until an "illegal address" fault is generated. This procedure defines the final legal address and hence the amount of available memory.

52 OS SERVICES

System Generation (SysGen)

- Information's to be determined by SysGen
  - What devices are available? The system will need to know how to address each device (the device number), the device interrupt number, the device's type and model, and any special device characteristics.
  - What operating-system options are desired, or what parameter values are to be used? These options or values might include how many buffers of which sizes should be used, what type of CPU-scheduling algorithm is desired, what the maximum number of processes to be supported is. …

53 OS SERVICES

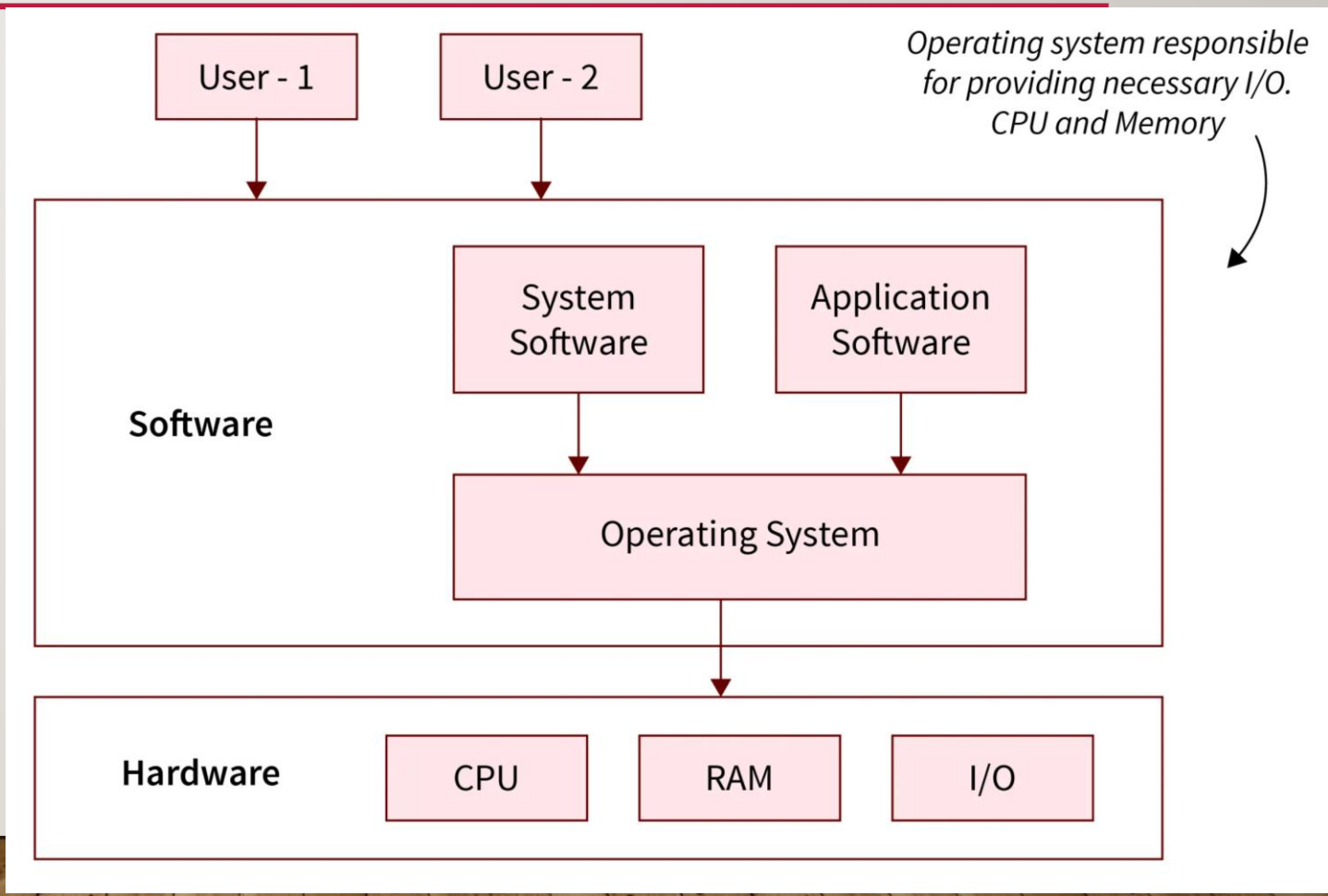
Booting

- The procedure of starting a computer by loading the kernel is known as booting the system.
- Most computer systems have a small piece of code, stored in ROM, known as the bootstrap program or bootstrap loader.
- This code is able to locate the kernel, load it into main memory, and start its execution.
- Some computer systems, such as PCs, use a two-step process in which a simple bootstrap loader fetches a more complex boot program from disk, which in turn loads the kernel.

COMPUTER SYSTEM ARCHITECTURE

55 COMPUTER SYSTEM ARCHITECTURE

- Computer system architecture in operating systems defines how hardware components (CPU, memory, I/O) are structured and managed to execute instructions efficiently.
- Main Functional Roles:
  - Process Management
  - Memory Management
  - I/O Management



56 COMPUTER SYSTEM ARCHITECTURE

Computer-System Operation

- I/O devices and the CPU can execute concurrently.
- Each device controller is in charge of a particular device type.
- Each device controller has a local buffer.
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller.
- Device controller informs CPU that it has finished its operation by causing an interrupt.

57 COMPUTER SYSTEM ARCHITECTURE

Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the interrupt vector, which contains the addresses of all the service routines.
- Interrupt architecture must save the address of the interrupted instruction.
- Incoming interrupts are disabled while another interrupt is being processed to prevent a lost interrupt.
- A trap is a software-generated interrupt caused either by an error or a user request.
- An operating system is interrupt driven.

58 COMPUTER SYSTEM ARCHITECTURE

Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter.
- Determines which type of interrupt has occurred: polling, vectored interrupt system?
- Separate segments of code determine what action should be taken for each type of interrupt.

59 COMPUTER SYSTEM ARCHITECTURE

Interrupt timeline for a Single Process doing Output

- Key Stages of the Interrupt Timeline:
  1. **Request**: The process issues an I/O output request (write to disk/screen). The CPU initiates the operation via a device driver.
  2. **Execution/Wait**: The I/O device starts transferring data. The process may be blocked (sleeps) waiting for completion, allowing other processes to run.
  3. **Interrupt Generation**: Upon finishing the data transfer, the I/O device controller sends an interrupt signal to the CPU.

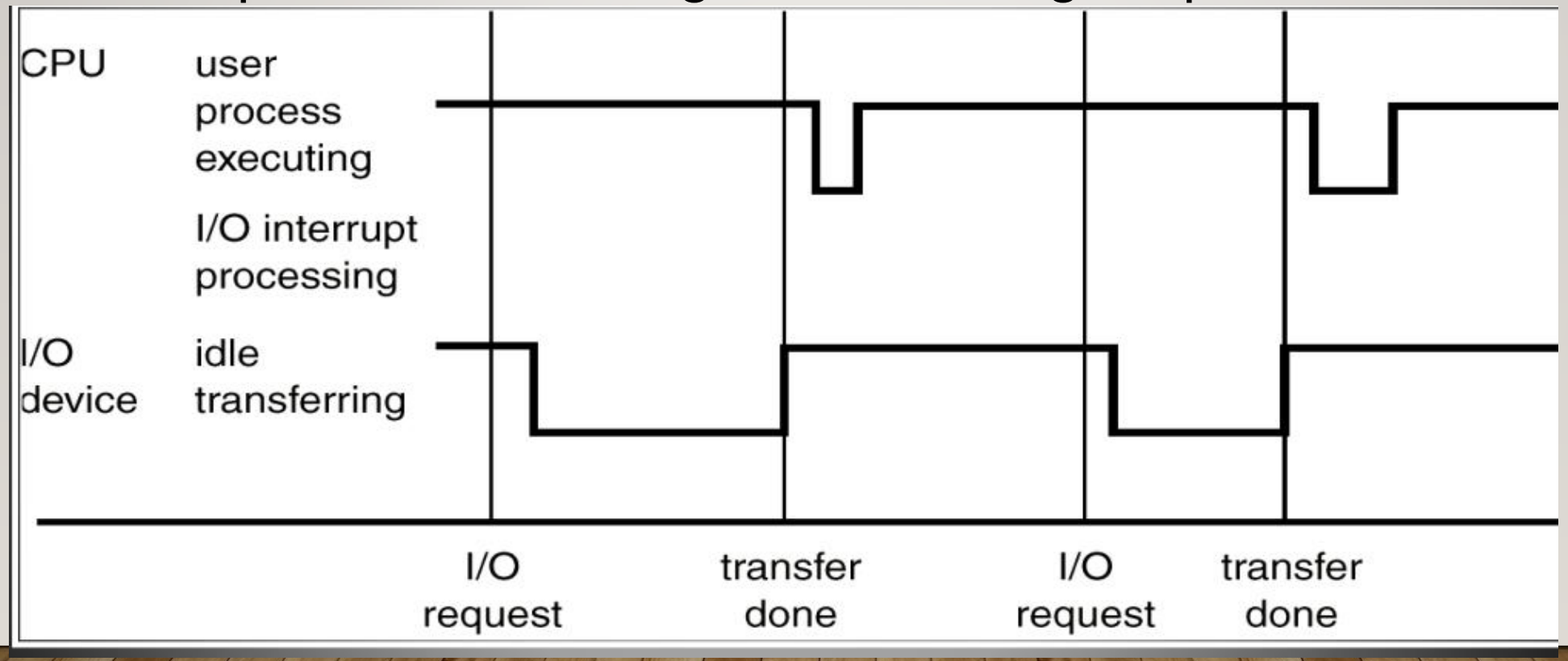
60 COMPUTER SYSTEM ARCHITECTURE

Interrupt timeline for a Single Process doing Output

- Key Stages of the Interrupt Timeline:
  4. **State Saving**: The CPU stops current work, saves the process state (registers, program counter) to memory, and identifies the interrupt.
  5. **Interrupt Service Routine (ISR)**: The CPU runs the ISR, which tells the OS that the I/O is done.
  6. **Resumption**: The CPU restores the saved state of the user process, which resumes execution immediately after the I/O call.

61 COMPUTER SYSTEM ARCHITECTURE

Interrupt timeline for a Single Process doing Output



62 COMPUTER SYSTEM ARCHITECTURE

I/O Structure

After I/O starts, control returns to user program only upon I/O completion.

- Wait instruction idles the CPU until the next interrupt
- Wait loop (contention for memory access).
- At most one I/O request is outstanding at a time, no simultaneous I/O processing.

After I/O starts, control returns to user program without waiting for I/O completion.

- System call – request to the operating system to allow user to wait for I/O completion.
- Device-status table contains entry for each I/O device indicating its type, address, and state.
- Operating system indexes into I/O device table to determine device status and to modify table entry to include interrupt.

63 COMPUTER SYSTEM ARCHITECTURE

Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds.
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention.
- Only on interrupt is generated per block, rather than the one interrupt per byte.

64 COMPUTER SYSTEM ARCHITECTURE

Storage Structure

- Main memory – only large storage media that the CPU can access directly.
- Secondary storage – extension of main memory that provides large nonvolatile storage capacity.
- Magnetic disks – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into tracks, which are subdivided into sectors.
  - The disk controller determines the logical interaction between the device and the computer.

COMPUTER SYSTEM ARCHITECTURE

Storage hierarchy

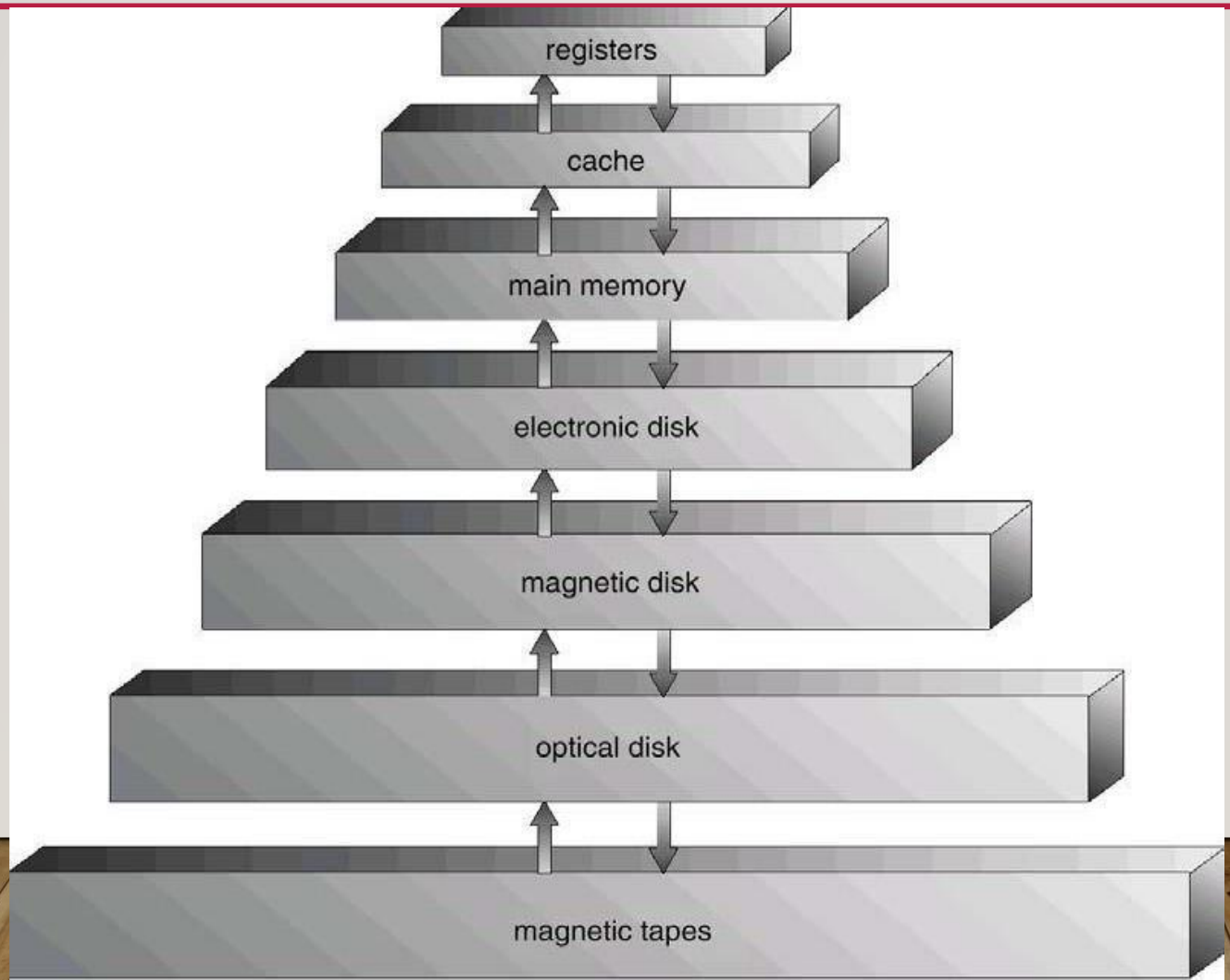
- Storage systems organized in hierarchy.
  - Speed
  - Cost
  - Volatility
- Caching – copying information into faster storage system; main memory can be viewed as a last cache for secondary storage.

COMPUTER SYSTEM ARCHITECTURE

Storage hierarchy

- Caching – copying information into faster storage system; main memory can be viewed as a last cache for secondary storage.
  - Use of high-speed memory to hold recently-accessed data.
  - Requires a cache management policy.
  - Caching introduces another level in storage hierarchy.
  - This requires data that is simultaneously stored in more than one level to be consistent.

67 COMPUTER SYSTEM ARCHITECTURE



68 COMPUTER SYSTEM ARCHITECTURE

Hardware Protection.

It is divided into 3 categories: CPU protection, Memory Protection, and I/O protection.

- **CPU protection** ensures that, a process does not monopolize the CPU indefinitely, as it would prevent other processes from being executed.
- In **memory protection**, we are talking about that situation when two or more processes are in memory, and one process may access the other process memory. To prevent this situation, we use two registers which are known as Base register and Limit register.

69 COMPUTER SYSTEM ARCHITECTURE

Hardware Protection

- **I/O protection**.
- With I/O protection, an OS ensures that following can be never done by a processes:
- Termination I/O of other process - This means one process should not be able to terminate I/O operation of other processes.
- View I/O of other process - One process should not be able to access the data being read/written by other processes from/to the Disk(s).
- Giving priority to a particular process I/O - No process must be able to prioritize itself or other processes which are doing I/O operations, over other processes.

70 COMPUTER SYSTEM ARCHITECTURE

Hardware Protection operations or mechanism

Dual-Mode Operation

- Sharing system resources requires operating system to ensure that an incorrect program cannot cause other programs to execute incorrectly.
- Provide hardware support to differentiate between at least two modes of operations.
  - User mode – execution done on behalf of a user.
  - Monitor mode (also kernel mode or system mode) – execution done on behalf of operating system.
- Mode bit added to computer hardware to indicate the current mode: monitor (0) or user (1).
- When an interrupt or fault occurs hardware switches to **Privileged instructions** can be issued only in monitor mode.

71 COMPUTER SYSTEM ARCHITECTURE

Hardware Protection operations or mechanism

- **I/O Protection**
  - All I/O instructions are privileged instructions.
  - Must ensure that a user program could never gain control of the computer in monitor mode.
- **Memory Protection**
  - Must provide memory protection at least for the interrupt vector and the interrupt service routines.
  - To have memory protection, add two registers that determine the range of legal addresses a program may access:
    - **Base register** – holds the smallest legal physical memory address.
    - **Limit register** – contains the size of the range
  - Memory outside the defined range is then protected.

72 COMPUTER SYSTEM ARCHITECTURE

Hardware Protection operations or mechanism

- **Hardware Protection**
  - When executing in monitor mode, the operating system has unrestricted access to both monitor and user's memory.
  - The load instructions for the base and limit registers are privileged instructions.

73 COMPUTER SYSTEM ARCHITECTURE

Hardware Protection operations or mechanism

- **CPU Protection**
  - Timer – interrupts computer after specified period to ensure operating system maintains control.
    - Timer is decremented every clock tick.
    - When timer reaches the value 0, an interrupt occurs.
  - Timer commonly used to implement time sharing.
  - Time also used to compute the current time.
  - Load-timer is a privileged instruction.

PROCESS CONCEPT

75 PROCESS CONCEPT

- A **process** is a program in execution.
- A process is more than the program code, which is sometimes known as the text section. It also includes the current activity, as represented by the value of the program counter and the contents of the processor's registers.
- A process generally includes the process stack, which contains temporary data (such as method parameters, return addresses, and local variables), and a data section, which contains global variables.

76 PROCESS CONCEPT

- **Process VS Program?**
  1. Both are same beast with different name or when this beast is sleeping (not executing) it is called program and when it is executing becomes process.
  2. Program is a static object whereas a process is a dynamic object.
  3. A program resides in secondary storage whereas a process resides in main memory.
  4. The span time of a program is unlimited, but the span time of a process is limited.
  5. A process is an 'active' entity whereas a program is a 'passive' entity.
  6. A program is an algorithm expressed in programming language whereas a process is expressed in assembly language or machine language.

77 PROCESS CONCEPT

- An operating system executes a variety of programs:
  - Batch system – jobs
  - Time-shared systems – user programs or tasks
- Process is a program in execution; process execution must progress in sequential fashion.
- A process include program counter, stack, data section

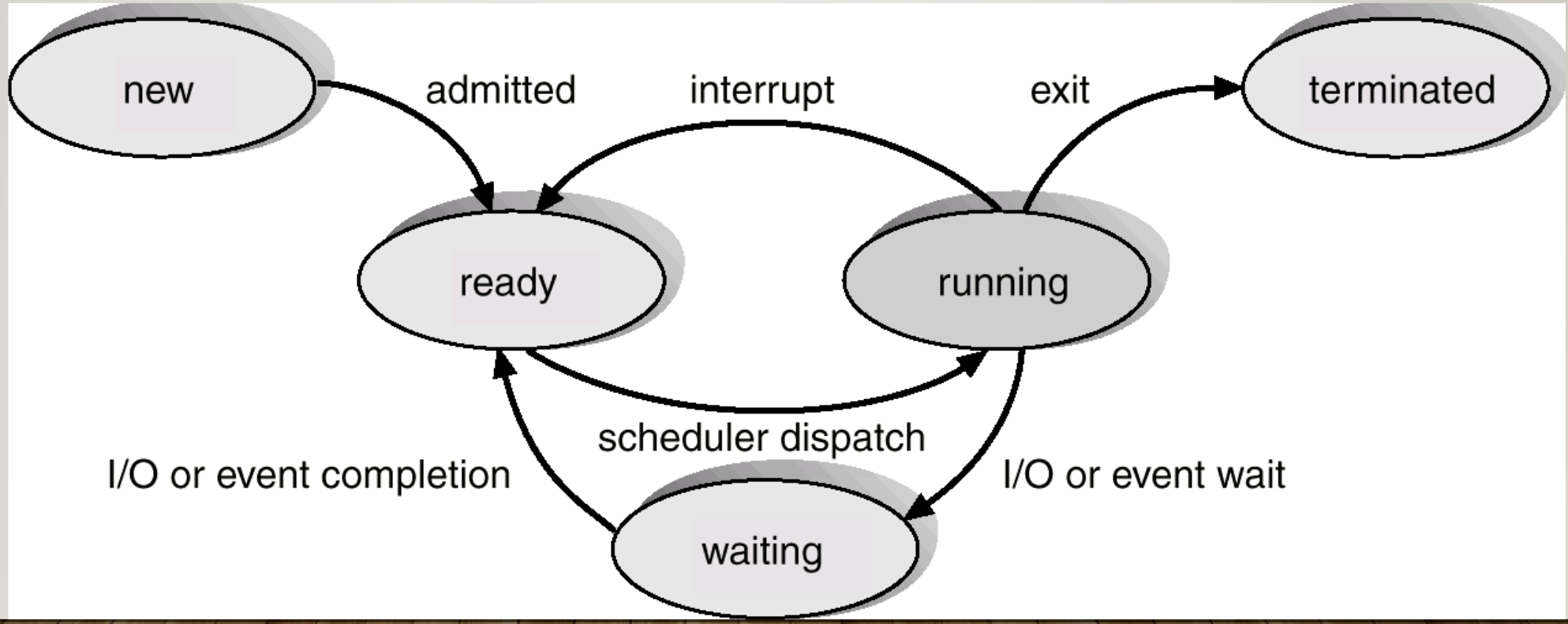
78 PROCESS CONCEPT

Process State

- As a process executes, it changes state. Different states are:
  - **New** State: The process is being created.
  - **Running** State: A process is said to be running if it has the CPU, process using the CPU at that instant.
  - **Blocked** (or **waiting**) State: A process is said to be blocked if it is waiting for some event to happen such as an I/O completion before it can proceed.
  - **Ready** State: A process is said to be ready if it needs a CPU to execute. A ready state process is runnable but temporarily stopped running to let another process run.
  - **Terminated** state: The process has finished execution.

79 PROCESS CONCEPT

Diagram of Process State

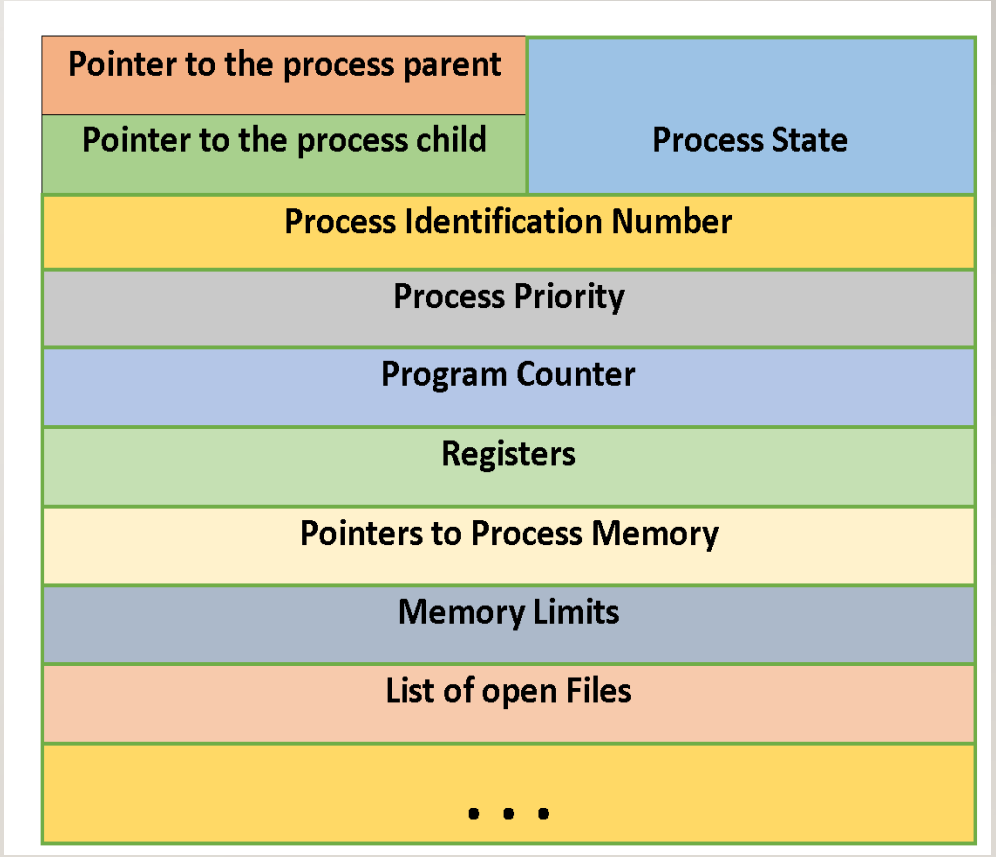


80 PROCESS CONCEPT

Process Control Block (PCB)

It contains information associated with each process. It simply serves as the repository for any information that may vary from process to process.

- **Process state**: The state may be new, ready, running, waiting, halted, …...
- **Program counter**: The counter indicates the address of the next instruction to be executed for this process.

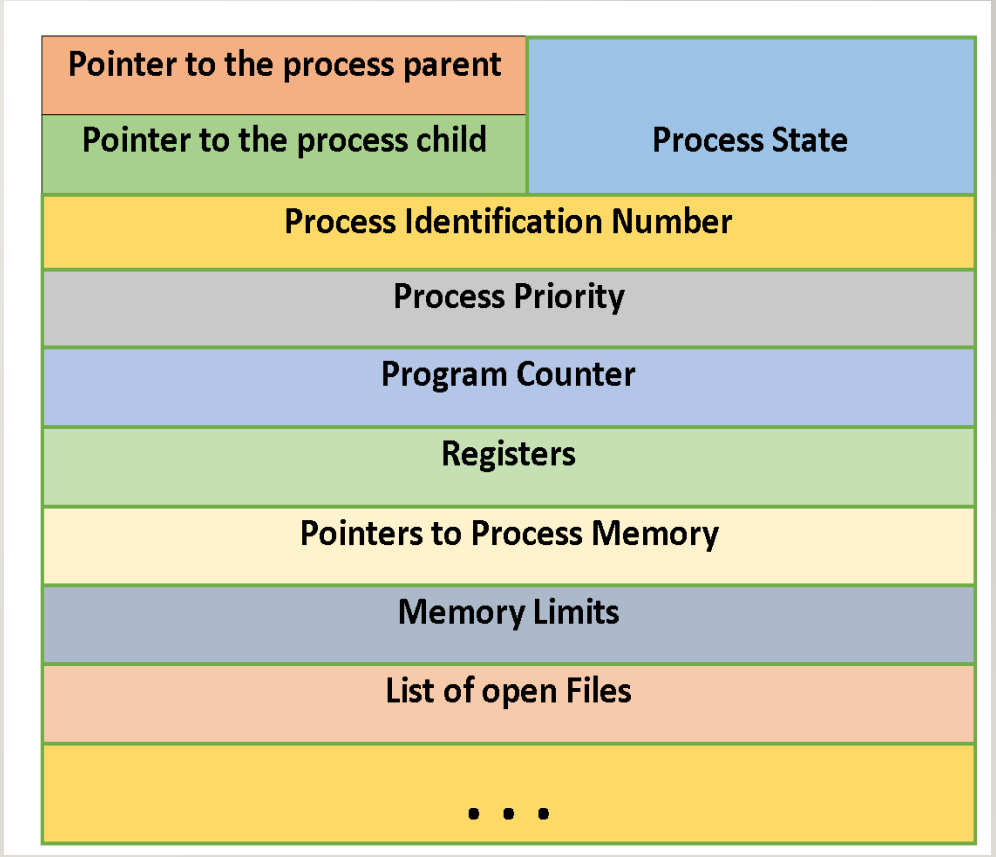


81 PROCESS CONCEPT

Process Control Block (PCB)

It contains information associated with each process.

- **CPU registers:** The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.

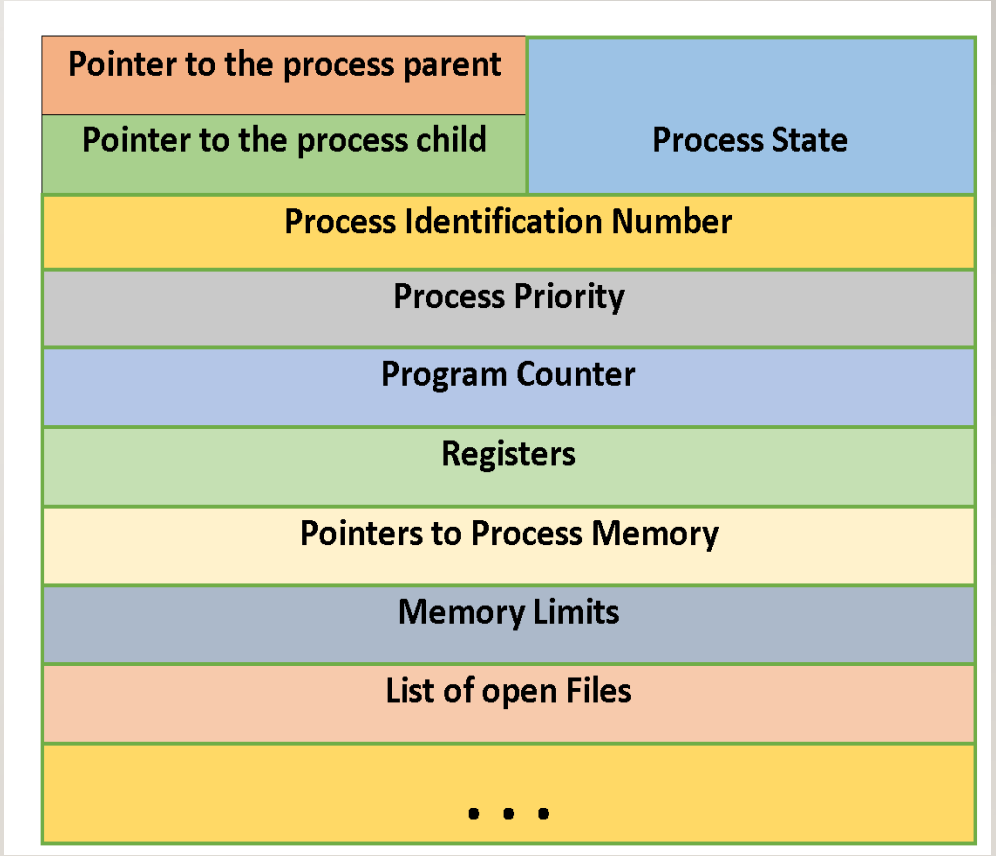


82 PROCESS CONCEPT

Process Control Block (PCB)

It contains information associated with each process.

- **CPU-scheduling information:** This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

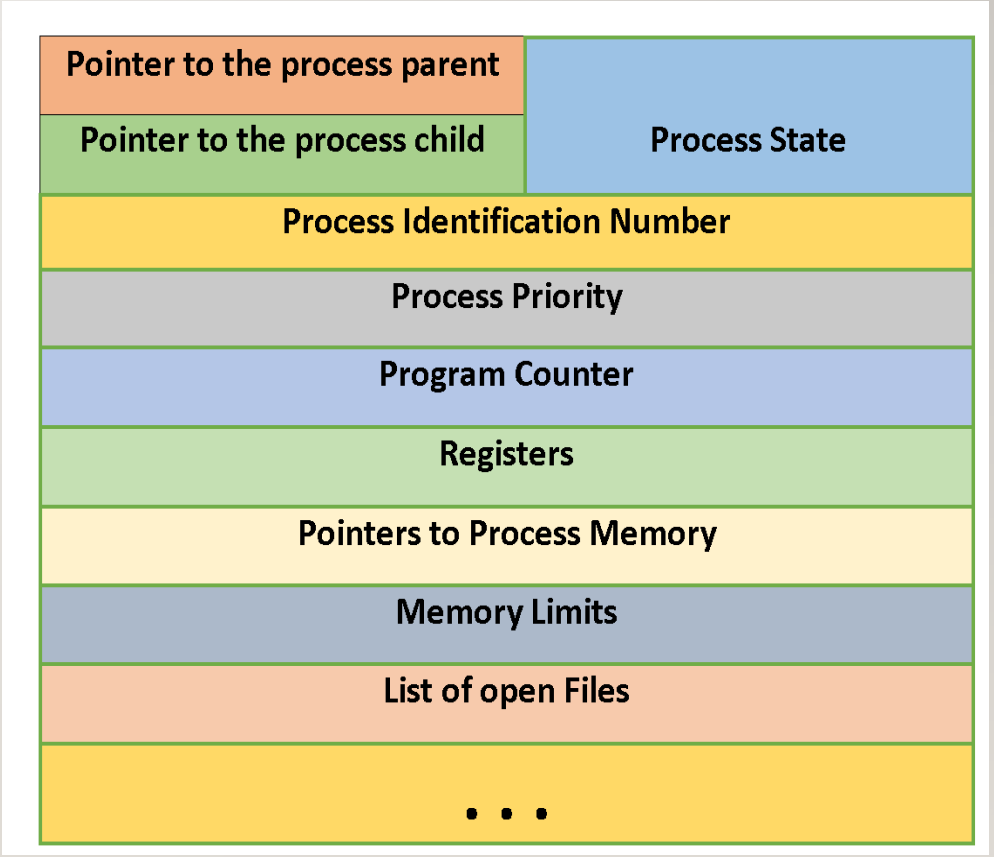


83 PROCESS CONCEPT

Process Control Block (PCB)

It contains information associated with each process.

- **Memory-management information:** This information may include such information as the value of the base and limit registers, the page tables, or the segment tables, depending on the memory system used by the operating system.

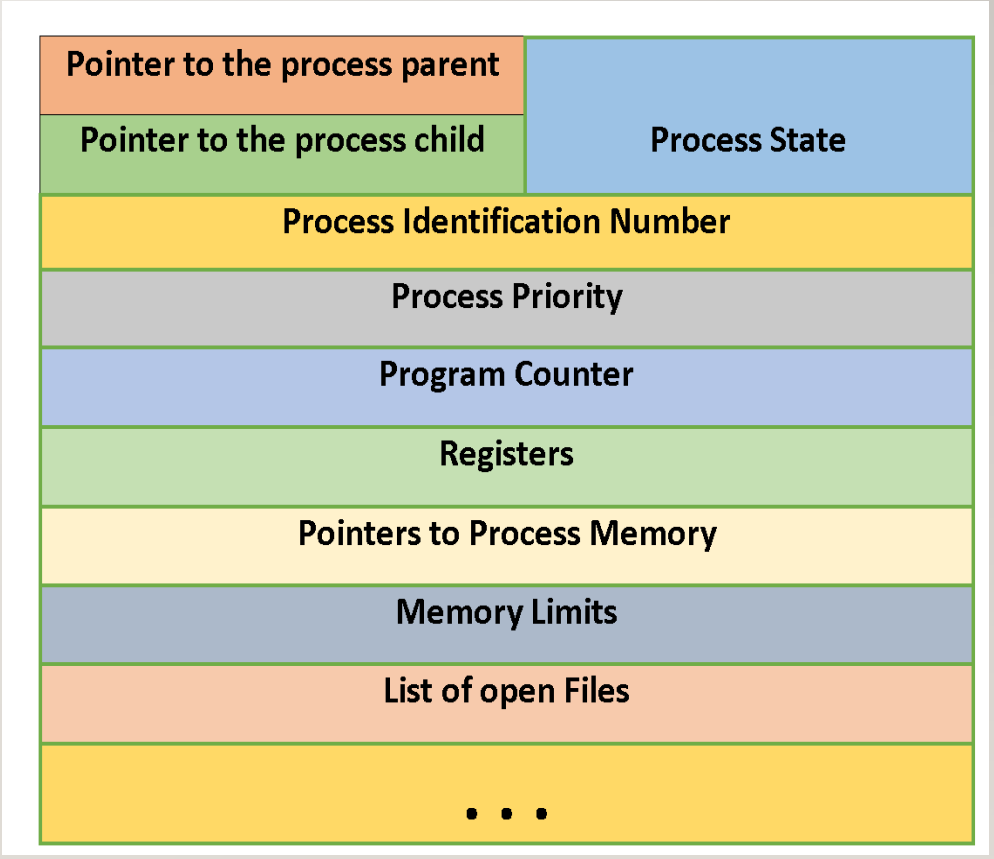


PROCESS CONCEPT

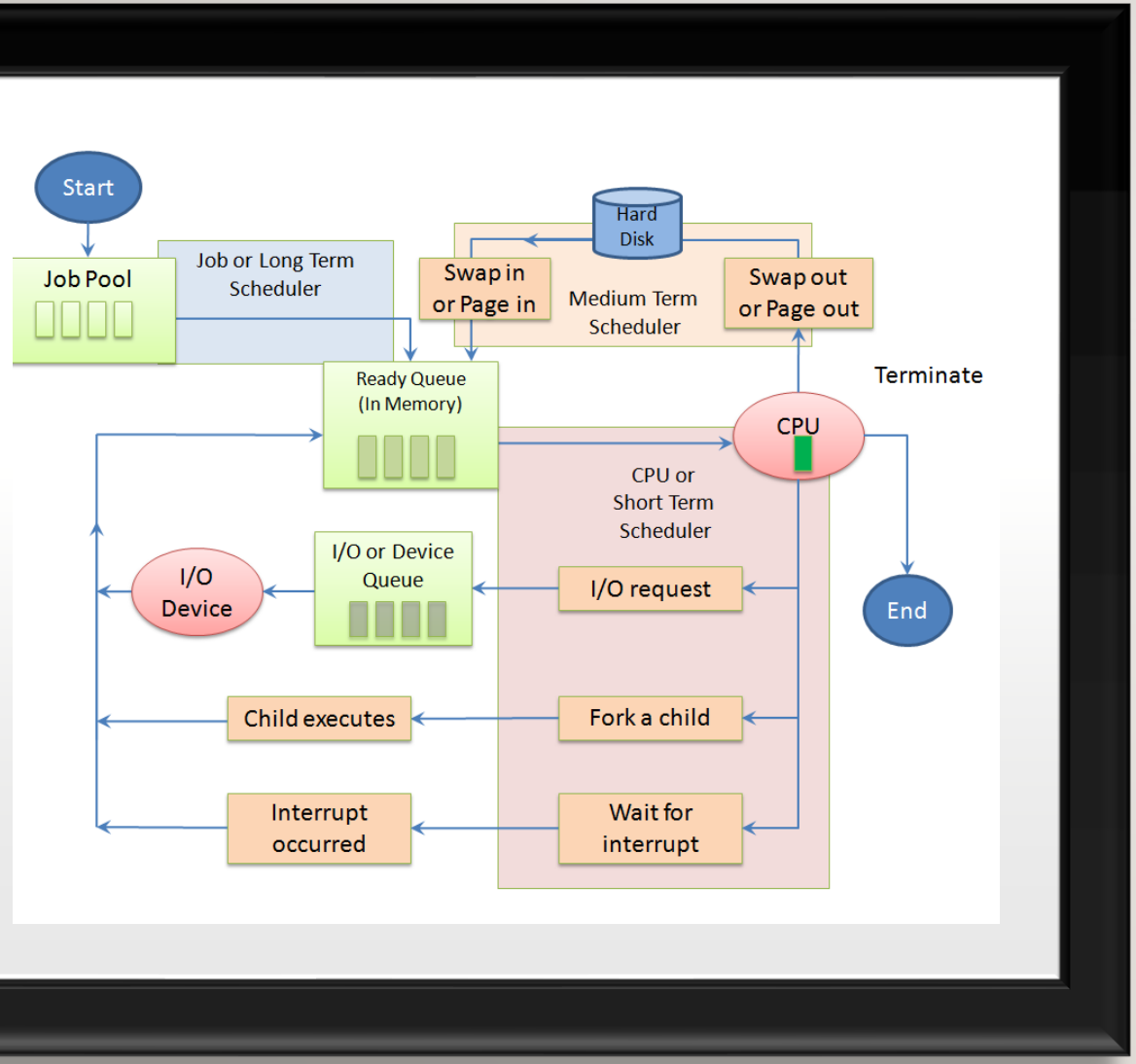
Process Control Block (PCB)

It contains information associated with each process.

- **Accounting information:** This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- **Status information:** The information includes the list of I/O devices allocated to this process, a list of open files, and so on.



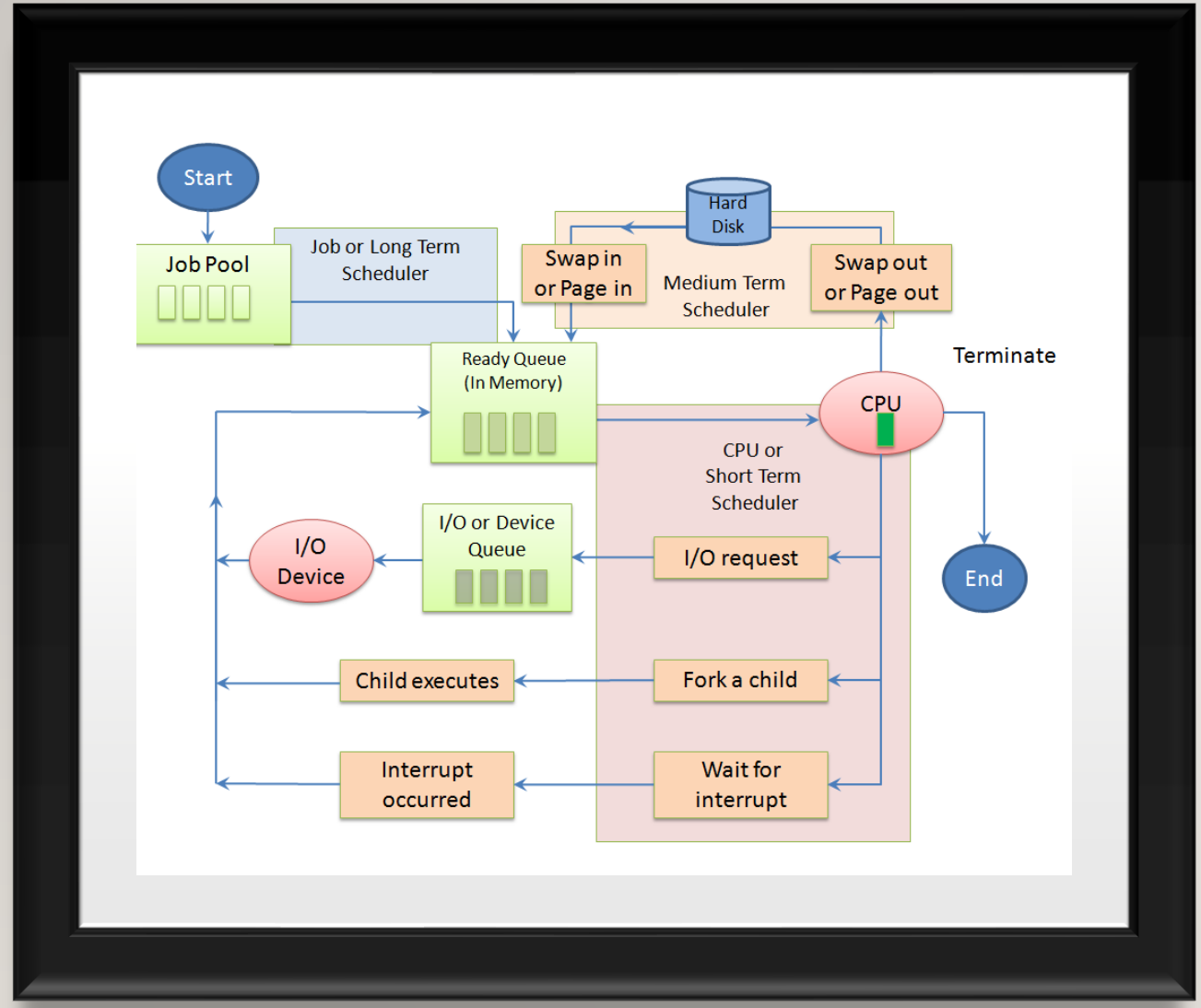
PROCESS CONCEPT



Process Scheduling Queue

- **Job Queue**: This queue consists of all processes in the system; those processes are entered to the system as new processes.
- **Device Queue**: This queue consists of the processes that are waiting for a particular I/O device. Each device has its own device queue.

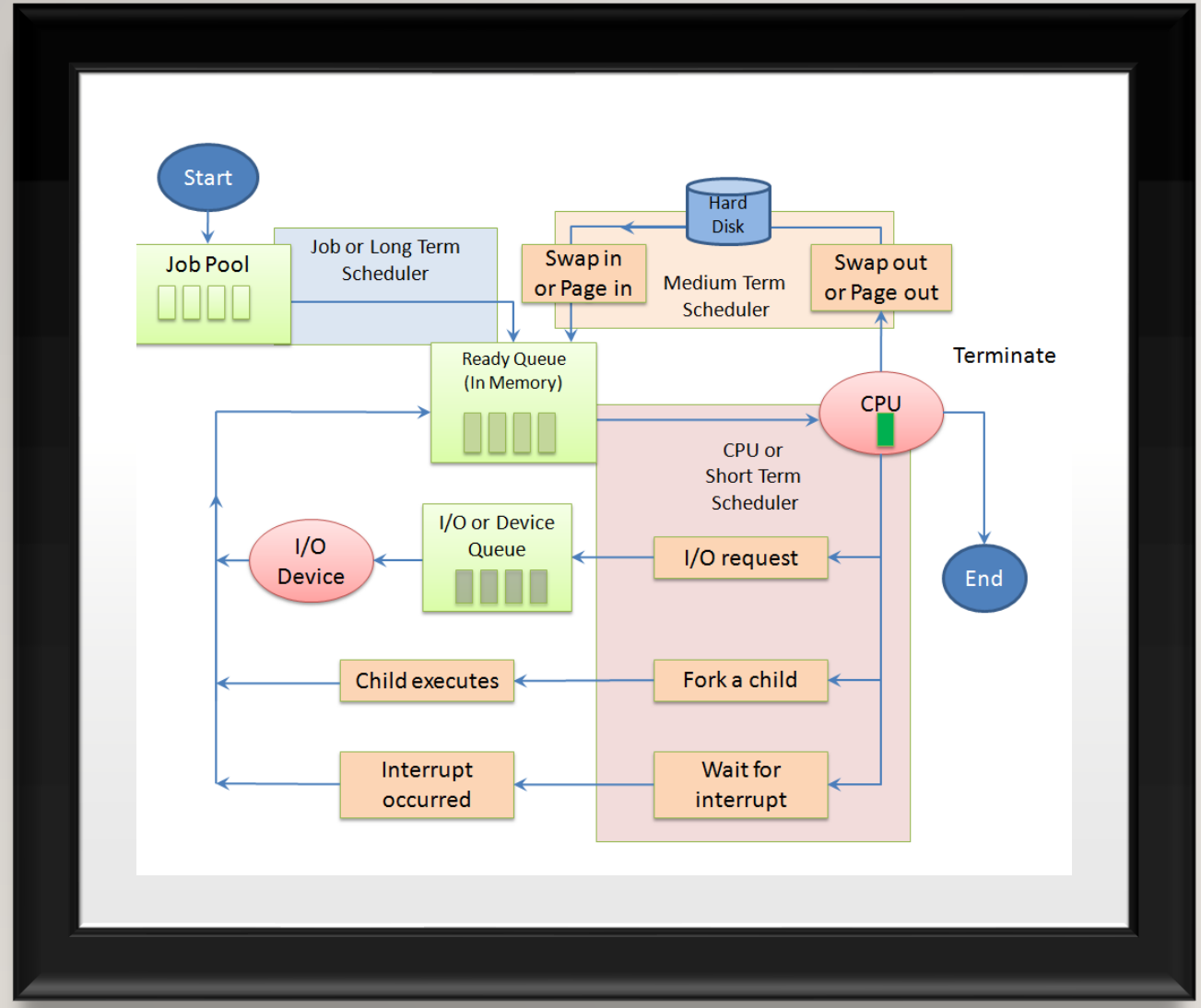
PROCESS CONCEPT



Process Scheduling Queue

- **Ready Queue**: This queue consists of the processes that are **residing in main memory** and are ready and waiting to execute by CPU. This queue is generally stored as a linked list. A ready-queue header contains pointers to the first and final PCBs in the list. Each PCB includes a pointer field that points to the next PCB in the ready queue.

PROCESS CONCEPT



Schedulers

- A scheduler is a decision maker that selects the processes from one scheduling queue to another or allocates CPU for execution.
- The Operating System has three types of scheduler:
  1. Long-term scheduler or Job scheduler
  2. Short-term scheduler or CPU scheduler
  3. Medium-term scheduler

PROCESS CONCEPT

Long-term scheduler or Job scheduler

- The long-term scheduler or job scheduler selects processes from discs and loads them into main memory for execution. It executes much less frequently.
- It controls the degree of multiprogramming (i.e., the number of processes in memory).
- Because of the longer interval between executions, the long-term scheduler can afford to take more time to select a process for execution.

Short-term scheduler or CPU scheduler

- The short-term scheduler or CPU scheduler selects a process from among the processes that are ready to execute and allocates the CPU.
- The short-term scheduler must select a new process for the CPU frequently. A process may execute for only a few milliseconds before waiting for an I/O request.

89 PROCESS CONCEPT

Medium-term scheduler

- The medium-term scheduler schedules the processes as intermediate level of scheduling Processes can be described as either:
  - I/O-bound process – spends more time doing I/O than computations, many short CPU bursts.
  - CPU-bound process – spends more time doing computations; few very long CPU bursts.
- Conclusion: A **scheduler** in an operating system is a **core component** that manages process execution by deciding which process runs on the CPU, when, and for how long. It maximizes CPU utilization and system throughput while minimizing response time by managing process states.

90 PROCESS CONCEPT

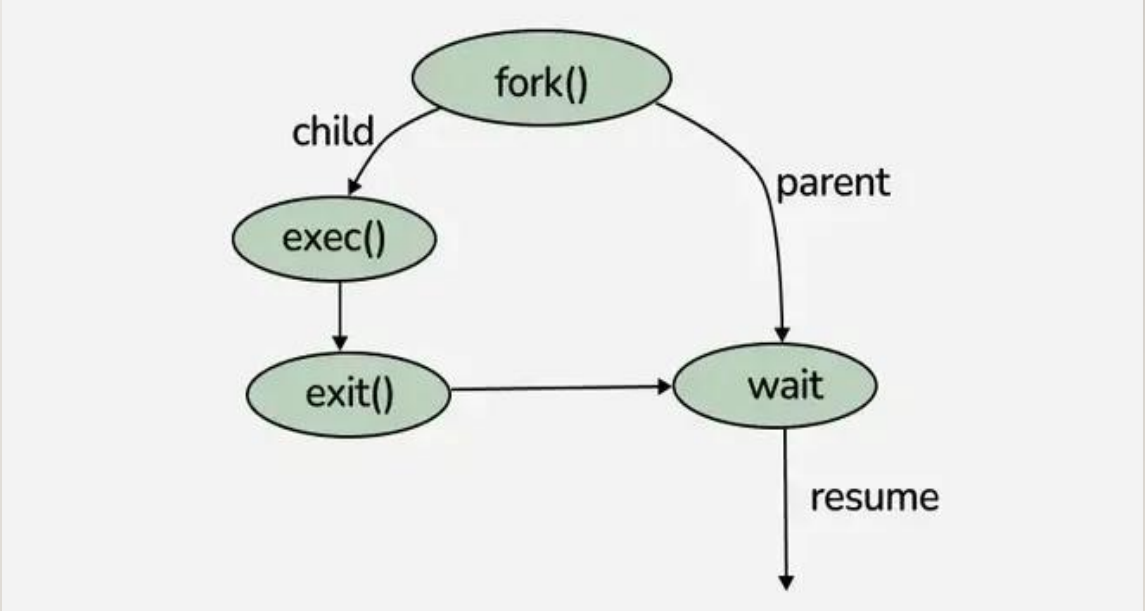
Context Switch

- Context switching is the process where an operating system saves the state (context) of a running process or thread—including registers, program counter, and memory maps—into a Process Control Block (PCB) and loads the saved state of a new process to resume execution.
- It enables multitasking, allowing multiple applications to appear as if they are running simultaneously.
- Context-switch time is overhead; the system does no useful work while switching.
- Time dependent on hardware support.

91 PROCESS CONCEPT

Process Creation

- Parent process create children processes, which, in turn create other processes, forming a tree of processes.
  - Resource sharing:
    - ✦ Parent and children share all resources.
    - ✦ Children share subset of parent's resources.
    - ✦ Parent and child share no resources.
  - Execution:
    - ✦ Parent and children execute concurrently.
    - ✦ Parent waits until children terminate.
  - Address space:
    - ✦ Child duplicate of parent.
    - ✦ Child has a program loaded into it.



92 PROCESS CONCEPT

Process Termination

- Process executes last statement and asks the operating system to decide it (exit).
  - Output data from child to parent (via wait).
  - Process' resources are deallocated by operating system.
- Parent may terminate execution of children processes (abort).
  - Child has exceeded allocated resources.
  - Task assigned to child is no longer required.
  - Parent is exiting.

✓Operating system does not allow child to continue if its parent terminates.

93 PROCESS CONCEPT

Cooperating Processes

- **Process is independent** if it cannot affect or be affected by the other processes executing in the system. Clearly, any process that does not share any data (temporary or persistent) with any other process is independent.
- **Process is cooperating** if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.
- The concurrent processes executing in the operating system **may be** either **independent** processes or **cooperating** processes.

94 PROCESS CONCEPT

Cooperating Processes Advantages

- **Information sharing**: Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to these types of resources.
- **Computation speedup**: If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Such a speedup can be achieved only if the computer has multiple processing elements (such as CPUS or I/O channels).

95 PROCESS CONCEPT

Cooperating Processes Advantages

- **Modularity:** We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads
- **Convenience:** Even an individual user may have many tasks on which to work at one time. For instance, a user may be editing, printing, and compiling in parallel.

Concurrent execution of cooperating processes requires mechanisms that allow processes to communicate with one another and to synchronize their actions.

CPU SCHEDULING

Definitions, algorithms and evaluations

97 CPU SCHEDULER

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Multiple-Processor Scheduling
- Real-Time Scheduling

98 CPU SCHEDULER

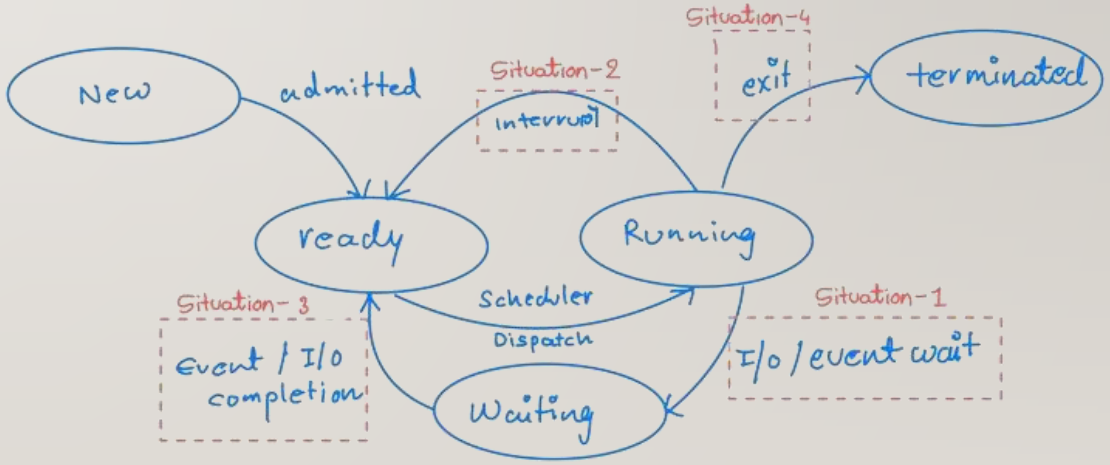
- CPU scheduling is the operating system process of selecting a waiting job from the ready queue and allocating the CPU to it, aiming to maximize CPU utilization, throughput, and efficiency while minimizing response and waiting times.
- It enables multitasking by switching the CPU among processes, using algorithms like FCFS, SJF, Round Robin, and Priority Scheduling

99 CPU SCHEDULER: KEY CONCEPTS

- **Ready Queue**: A collection of processes (often PCB records) present in main memory that are ready and waiting to execute.
- **CPU Scheduler (Short-term)**: Selects a process from the ready queue to run.
- **Dispatcher**: Gives control of the CPU to the process selected by the scheduler.
- **Preemptive Scheduling**: The OS can interrupt a running process to allocate the CPU to another process, often based on priority or time-sharing.
- **Non-Preemptive Scheduling**: A running process keeps the CPU until it voluntarily releases it (terminates or waits for I/O)

100 CPU SCHEDULER

- CPU Scheduler
  - Selects from among the processes in memory that are ready to execute and allocates the CPU to one of them.
  - CPU scheduling decisions may take place when a process:
    1. Switches from running to waiting state.
    2. Switches from running to ready state.
    3. Switches from waiting to ready.
    4. Terminates.
  - Scheduling under 1 and 4 is non preemptive.
  - All other scheduling is preemptive.



101 CPU SCHEDULER

- **Dispatcher**: a module that gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - Switching context
  - Switching to user mode
  - Jumping to the proper location in the user program to restart that program
- **Dispatch latency**: time it takes for the dispatcher to stop one process and start another running.

102 CPU SCHEDULER: SCHEDULING CRITERIA

- **CPU Utilization**: Keeping the CPU as busy as possible (in real systems).
- **Throughput**: Number of processes completed per unit of time.
- **Turnaround Time**: Total time taken from submission to completion of a process.
- **Waiting Time**: Total time a process spends waiting in the ready queue.
- **Response Time**: Time taken from submission until the first response is produced (critical in interactive systems)

103 CPU SCHEDULER: CHALLENGES

- **Starvation**: Low-priority processes may never run if higher-priority processes constantly arrive.
- **Synchronization**: Ensuring consistency when multiple processes access shared resources.
- **Algorithm Selection**: Choosing the best algorithm depends on the system's specific performance goals

104 CPU SCHEDULER: OPTIMIZATION CRITERIA

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

105 CPU SCHEDULER: ALGORITHMS

- **First-Come, First-Served (FCFS)**: Simplest algorithm, tasks are executed in the order they arrive. It is non-preemptive.
- **Shortest Job First (SJF)**: Selects the process with the smallest expected CPU burst time.
- **Shortest Remaining Time First (SRTF)**: The preemptive version of SJF, which selects the process with the shortest remaining burst time.

106 CPU SCHEDULER: ALGORITHMS

- **Round Robin (RR)**: Each process gets a fixed time slot (quantum) in a circular queue, providing fair, interactive access.
- **Priority Scheduling**: Assigns a priority to each process; the CPU is allocated to the process with the highest priority.
- **Multilevel Queue Scheduling**: Processes are divided into separate queues based on characteristics (e.g., system processes, interactive processes), often with specific algorithms for each.

107 CPU SCHEDULER: ALGORITHMS

- **First Come, First served**
  - By far the simplest CPU-scheduling algorithm
  - The process that requests the CPU first is allocated the CPU first.
  - The FCFS policy is easily managed with a FIFO queue.
  - When a process enters the ready queue, its PCB is linked onto the tail of the queue.
  - When the CPU is free, it is allocated to the process at the head of the queue.
  - The running process is then removed from the queue.

CPU SCHEDULER: ALGORITHMS

- **First Come, First served**
  - **Example**
    - **In the first table, all arrived at time 0**
    - **But the arrived is in an order**
    - **Each process has a duration (time to use the CPU)**

Process	Order	Duration	Arrival time
P1	1	24	0
P2	2	3	0
P3	3	4	0

Process	Order	Duration	Arrival time
P1	1	24	1
P2	2	3	2
P3	3	4	0

109 CPU SCHEDULER: ALGORITHMS

- **First Come, First served**
  - **Example:**
    - If the processes arrive in the order P_2, P_3, P_1, ,
    - Draw the Gannt chart
    - Compute waiting time and average waiting time

110 CPU SCHEDULER: ALGORITHMS

- **First Come, First served**
  - **Advantage:**
    - Easy to understand,
    - simple implementation,
    - fair in terms of arrival.
  - **Inconvenience:**
    - Poor efficiency,
    - long average waiting times,
    - vulnerable to the convoy effect

CPU SCHEDULER: ALGORITHMS

- Assume FCFS scheduling in a dynamic situation, with one CPU-bound process and many I/O-bound processes.
- The following scenario may result.
  - The CPU-bound process will get the CPU and hold it.
  - During this time, all the other processes will finish their I/O and move into the ready queue, waiting for the CPU.
  - While the processes wait in the ready queue, the I/O devices are idle.
  - Eventually, the CPU-bound process finishes its CPU burst and moves to an I/O device.
  - All the I/O-bound processes, which have very short CPU bursts, execute quickly and move back to the I/O queues. At this point, the CPU sits idle.

112 CPU SCHEDULER: ALGORITHMS

Assume FCFS scheduling in a dynamic situation, with one CPU-bound process and many I/O-bound processes. The following scenario may result.

- The CPU-bound process will then move back to the ready queue and be allocated the CPU.
- All the I/O processes end-up waiting in the ready queue until the CPU-bound process is done.
- There is a convoy effect, as all the other **processes wait** for the one big process to get off the CPU.
- This effect results in **lower CPU and device utilization** than might be possible if the shorter processes were allowed to go first.

113 CPU SCHEDULER: ALGORITHMS

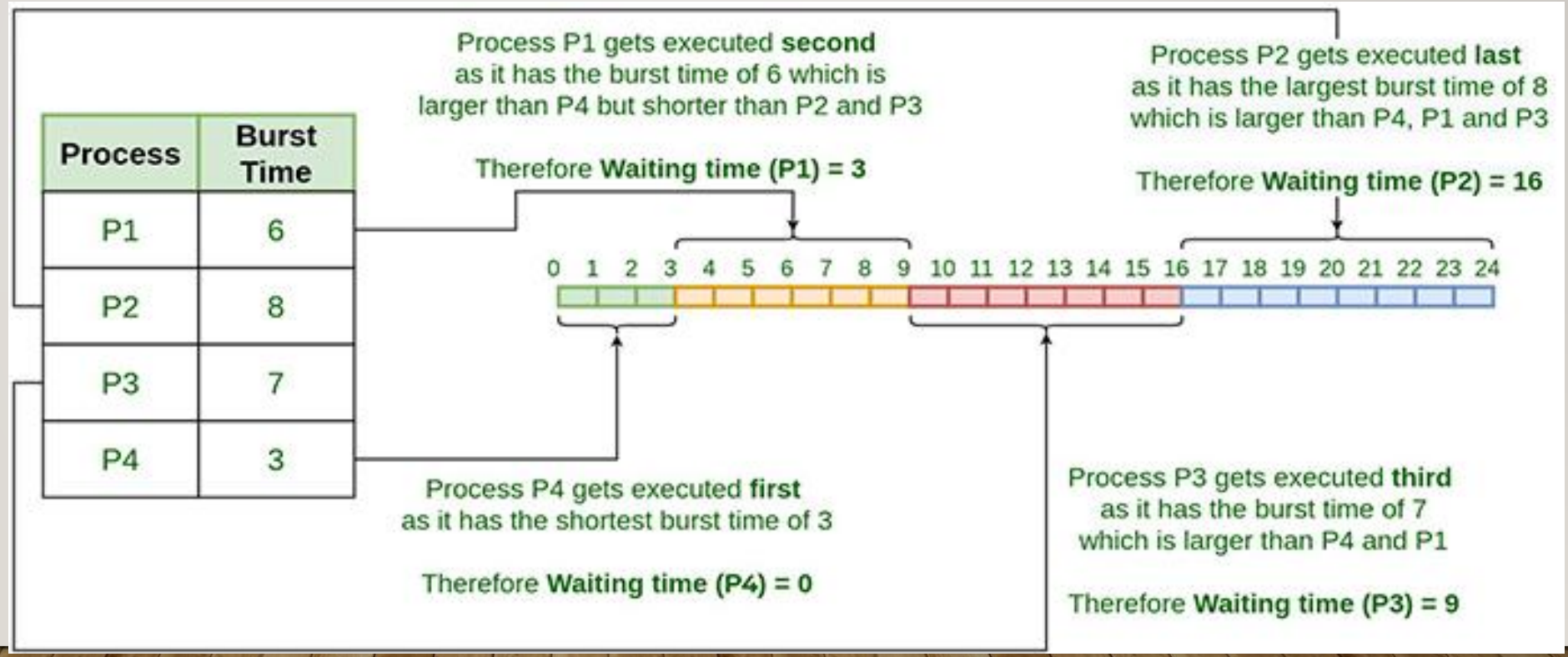
- **First Come, First served**
  - **Characteristics of FCFS**
    - **Non-Preemptive**: Once a process is assigned the CPU, it holds it until it voluntarily releases it (finishes or requests I/O).
    - **Arrival Order**: The process with the minimum arrival time is executed first.
    - **No Starvation**: Since every process eventually gets a turn, starvation does not occur.

114 CPU SCHEDULER: ALGORITHMS

- **Shortest-Job-First Scheduling**
  - This algorithm associates with each process the length of the **latter's next CPU burst**.
  - When the CPU is available, it is assigned to the process that has the smallest next CPU burst.
  - If two processes have the same length of next CPU burst, FCFS scheduling is used to break the tie.
  - A more appropriate term would be the **shortest next CPU burst**, because the scheduling is done by examining the length of the **next CPU burst** of a process, rather than **its total length**.
  - SJF can be **non-preemptive** or **preemptive**

115 CPU SCHEDULER: ALGORITHMS

- **Shortest-Job-First Scheduling** (non-preemptive)
  - Example



116 CPU SCHEDULER: ALGORITHMS

- **Shortest-Job-First Scheduling**
  - Performances
    - Order of executions: P4, P1, P3, P2
    - Waiting Time: P4=0; P1=3, P3= 9 and P2=16
    - Average Waiting time (0+3+9+16)/4= 7

117 CPU SCHEDULER: ALGORITHMS

- **Shortest-Job-First Scheduling**
  - The SJF scheduling algorithm is provably optimal, in that it gives the minimum average waiting time for a given set of processes.
    - Consequently, the average waiting time decreases.
  - The **real difficulty** with the SJF algorithm is knowing the length of the next CPU request.
    - For long-term scheduling (in a batch system), we can use this method, as the length the process time limit that a user specifies when he submits the job.

118 CPU SCHEDULER: ALGORITHMS

- **Shortest-Job-First Scheduling**
  - Although the SJF algorithm is optimal, it cannot be implemented at the level of short-term CPU scheduling.
    - Why? There is no way to know the length of the next CPU burst.

119 CPU SCHEDULER: ALGORITHMS

- **Shortest-Job-First Scheduling** (preemptive)
  - The choice arises when a new process arrives at the ready queue while a previous process is executing.
  - The new process may have a shorter next CPU burst than what is left of the currently executing process.
  - A preemptive SJF algorithm will **preempt** the currently executing process, whereas a non-preemptive SJF algorithm will allow the currently running process to finish its CPU burst.
  - Preemptive SJF scheduling is sometimes called **shortest-remaining-time-first** scheduling.

CPU SCHEDULER: ALGORITHMS

- **Shortest-Job-First Scheduling**
  - Example
    - Arrival times (known)
    - CPU Burst Time (known)
    - Draw the Gannt chart
    - Compute the
      - Completion time (CT)
      - Turn around time (TAT)
      - Waiting time (2 methods) (WT)
      - Avg-WT
      - Avg-TAT

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling**
  - A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
  - Equal-priority processes are scheduled in FCFS order.
  - The SJF algorithm is a special case of the general priority-scheduling algorithm.
  - The larger the CPU burst, the lower the priority, and vice versa.
  - Some systems use low numbers to represent low priority; others use low numbers for high priority.
  - We use **low numbers** to represent **high priority**.

122 CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling**
  - Priority scheduling is one of the most common scheduling algorithms used by the operating system to schedule processes based on their priority.
  - Each process is assigned a priority value based on criteria such as memory requirements, time requirements, other resource needs, or the ratio of average I/O to average CPU burst time.
  - The process with the highest priority is selected for execution first.

123 CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling**
  - If there are multiple processes sharing the same priority, they are scheduled in the order they arrived, following a First-Come, First-Served approach.
  - The chosen process is then executed, either **until completion** or until it is **preempted**, depending on whether the scheduling is preemptive or non-preemptive.

CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling: Non-Preemptive**
  - In Non-Preemptive Priority Scheduling, the CPU is not taken away from the running process.
    - Even if a higher-priority process arrives, the currently running process will complete first.
  - Example:

Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	2	1
P3	2	6	3

125 CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling: Non-Preemptive**
- **Gantt chart:**

P1 (0-4)	P2 (4-6)	P3 (6-12)

- **Average waiting time?**
  - CT(P1)= 4;WT(P1)= 0;TAT(P1)=4.
  - CT(P2)=6.WT(P2)=3;TAT(P2)=5.
  - CT(P3)=12;WT(P3)=4 ;TAT=10.
  - ATAT= 19/3=6.33
  - AWT=7/3=2.33

Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	2	1
P3	2	6	3

126 CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling: Preemptive**
  - In Preemptive Priority Scheduling, the CPU can be taken away from the currently running process if a new process with **a higher priority arrives**.
  - Example 1: same arrival time
    - ATAT=10.66
    - AWT=5

Process	Arrival Time	Burst Time	Priority
P1	0	7	2
P2	0	4	1
P3	0	6	3

CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling: Preemptive**
  - In Preemptive Priority Scheduling, the CPU can be taken away from the currently running process if a new process with **a higher priority arrives**.
  - Example 2: Different arrival time
    - ATAT= 9
    - AWT= 4.66

Process	Arrival Time	Burst Time	Priority
P1	0	6	2
P2	1	4	1
P3	2	5	3

CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling: Preemptive**
  - Exercise
    - FCFS
    - SJF non-preemptive
    - SJF preemptive
    - Priority non preemptive
    - Priority preemptive
  - ATAT= ?
  - AWT= ?

Process	Arrival Time	Burst Time	Priority
P1	0	10	3
P2	1	1	1
P3	2	2	4
P4	4	1	5
P5	5	5	2

129 CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling**
  - A major problem with priority-scheduling algorithms is indefinite blocking (or starvation).
  - A process that is ready to run but lacking the CPU can be considered blocked-waiting for the CPU.
  - A priority-scheduling algorithm can leave some low-priority processes waiting indefinitely for the CPU.

130 CPU SCHEDULER: ALGORITHMS

- **Priority Scheduling**
  - A solution to the problem of indefinite blockage of low-priority processes is **aging**.
  - Aging is a technique of gradually increasing the priority of processes that wait in the system for a long time.
  - Example
    - if priorities range from 120 (low) to 0 (high), we could decrement the priority of a waiting process by 1 every 5 minutes.
    - Eventually, even a process with an initial priority of 120 would have the highest priority in the system and would be executed.

131 CPU SCHEDULER: ALGORITHMS

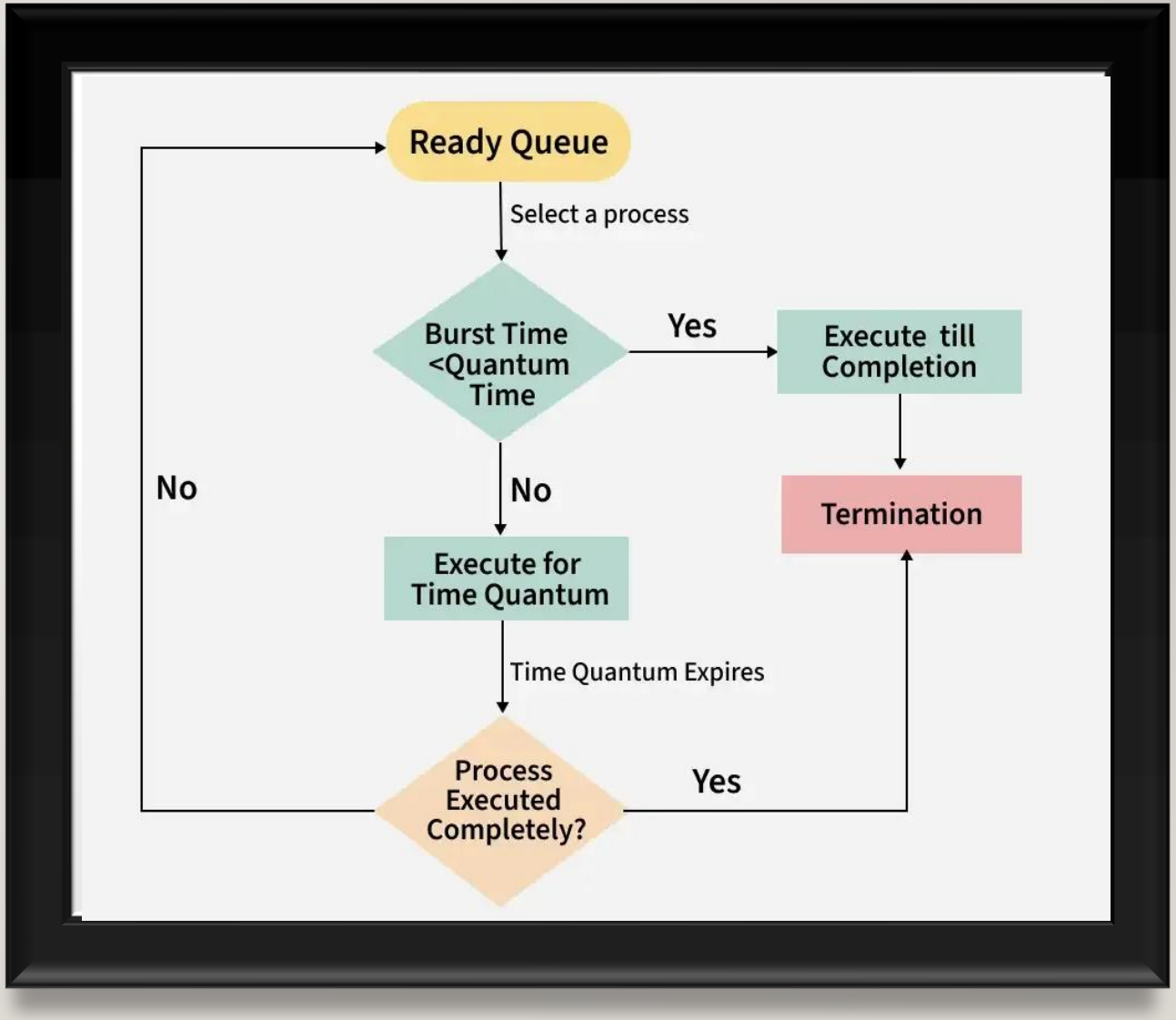
- **Round Robin Scheduling**
  - It is a method used by operating systems to manage the execution time of multiple processes that are competing for CPU attention.
  - It is called "round robin" because the system rotates through all the processes, allocating each of them a fixed time slice or "quantum", regardless of their priority.
  - The primary goal of this scheduling method is to ensure that all processes are given an equal opportunity to execute, promoting fairness among tasks.

132 CPU SCHEDULER: ALGORITHMS

- **Round Robin Scheduling**
  - **Process Arrival**: Processes enter the system and are placed in a queue.
  - **Time Allocation**: Each process is given a certain amount of CPU time, called a quantum.
  - **Execution**: The process uses the CPU for the allocated time.
  - **Rotation**: If the process completes within the time, it leaves the system. If not, it goes back to the end of the queue.
  - **Repeat**: The CPU continues to cycle through the queue until all processes are completed.

CPU SCHEDULER: ALGORITHMS

- **Round Robin Scheduling**



134 CPU SCHEDULER: ALGORITHMS

- **Round Robin Scheduling**
  - **Example** All arrived at the same time
  - Quantum time is 2
  - Gantt chart is

Process	AT	BT
P1	0	4
P2	0	5
P3	0	3

P1 (0-2)	P2 (2-4)	P3 (4-6)	P1 (6-8)	P2 (8-10)	P3 (10-11)	P2 (11-12)

  - Turnaround Time = Completion Time - Arrival Time.
    - ATAT= (8 + 12 + 11)/3 = 31/3 = 10.33
  - Waiting Time = Turnaround Time - Burst Time
    - AWT = (4 + 7 + 8)/3 = 19/3 = 6.33

135 CPU SCHEDULER: ALGORITHMS

- **Round Robin Scheduling**
  - **Example** Process arrived at different time
  - Quantum time is 2
  - Gantt chart is

P1 (0-2)	P1 (2-4)	P2 (4-6)	P1 (6-7)	P3 (7-9)	P3 (2-11)

  - Turnaround Time = Completion Time - Arrival Time.
    - ATAT= (7+2+6)/3=15/3=5
  - Waiting Time = Turnaround Time - Burst Time
    - AWT = (2+0+2)/3=1.33

Process	AT	BT
P1	0	5
P2	4	2
P3	5	4

136 CPU SCHEDULER: ALGORITHMS

- **Round Robin Scheduling: Advantages**
  - **Fairness**: Each process gets an equal share of the CPU.
  - **Simplicity**: The algorithm is straightforward and easy to implement.
  - **Responsiveness**: Round Robin can handle multiple processes without significant delays, making it ideal for time-sharing systems.

137 CPU SCHEDULER: ALGORITHMS

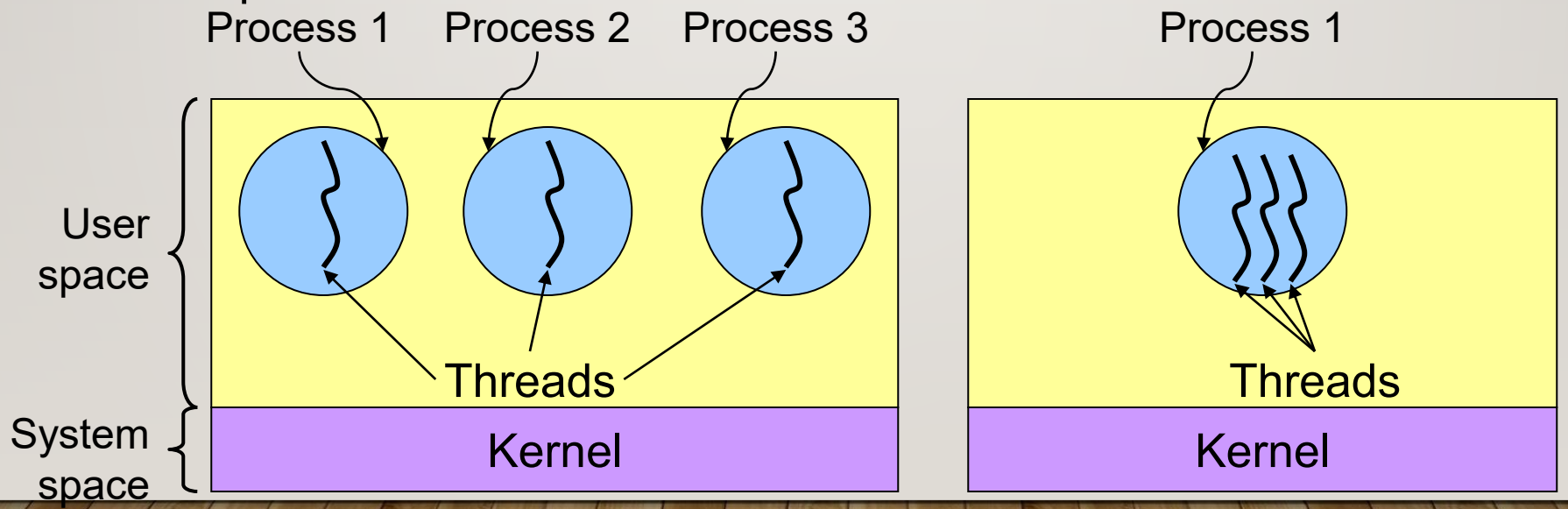
- **Round Robin Scheduling: Disadvantages**
  - **Overhead**: Switching between processes can lead to high overhead, especially if the quantum is too small.
  - **Underutilization**: If the quantum is too large, it can cause the CPU to feel unresponsive as it waits for a process to finish its time.

138 THREADS

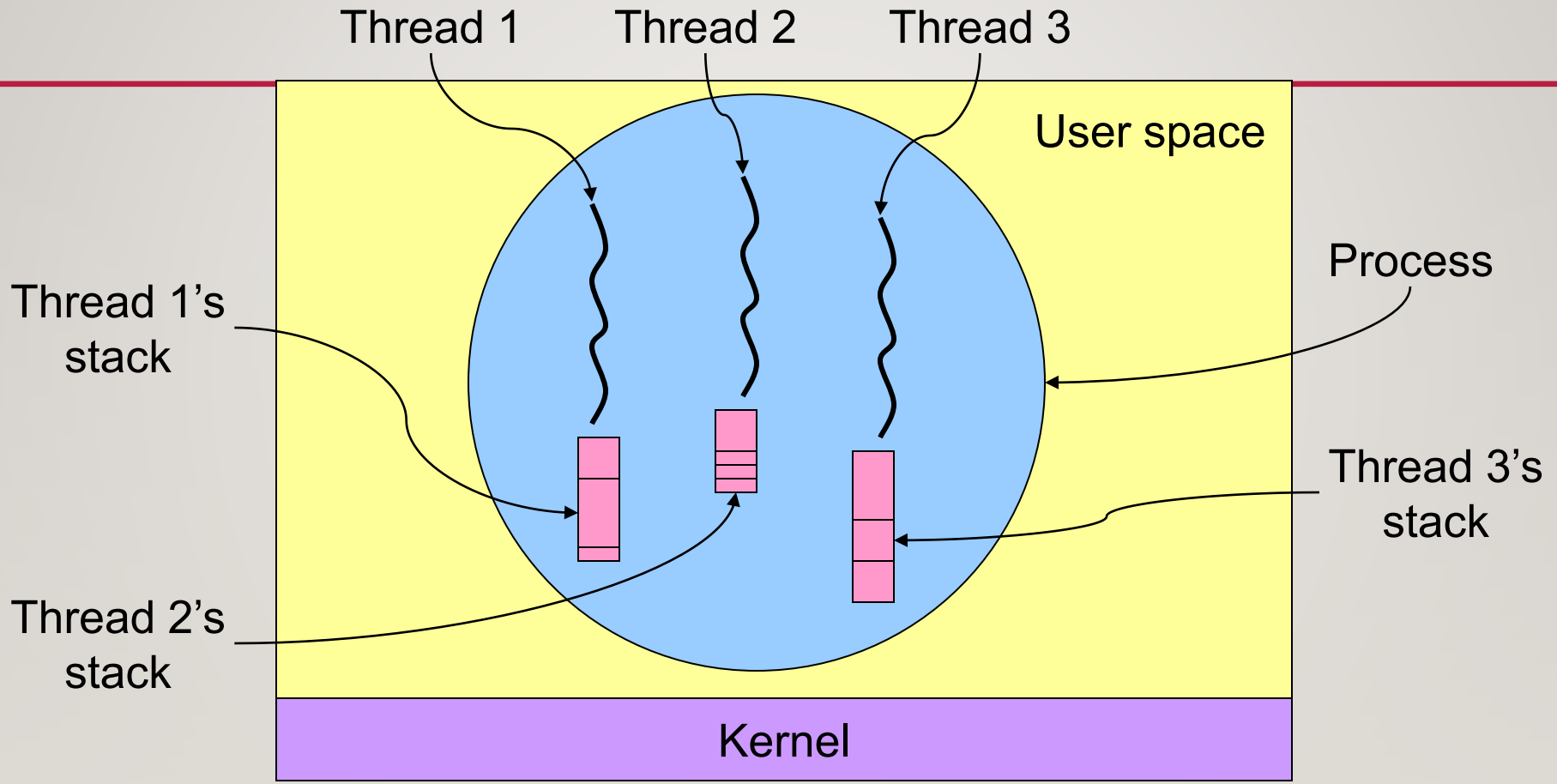
- A thread is the smallest unit of execution within an operating system process, often called a "lightweight process - LWP".
- Threads share memory and resources of their parent process (code, data, files) but have their own program counter, stack, and register set.
- They enable parallelism and improve application responsiveness, allowing tasks like audio and video to run concurrently.
- It is then a basic unit of CPU utilization.

THREADS: PROCESSES SHARING MEMORY

- Process == address space
- Thread == program counter / stream of instructions
- Two examples
  - Three processes, each with one thread
  - One process with three threads



140 THREADS & STACKS



=> Each thread has its own stack!

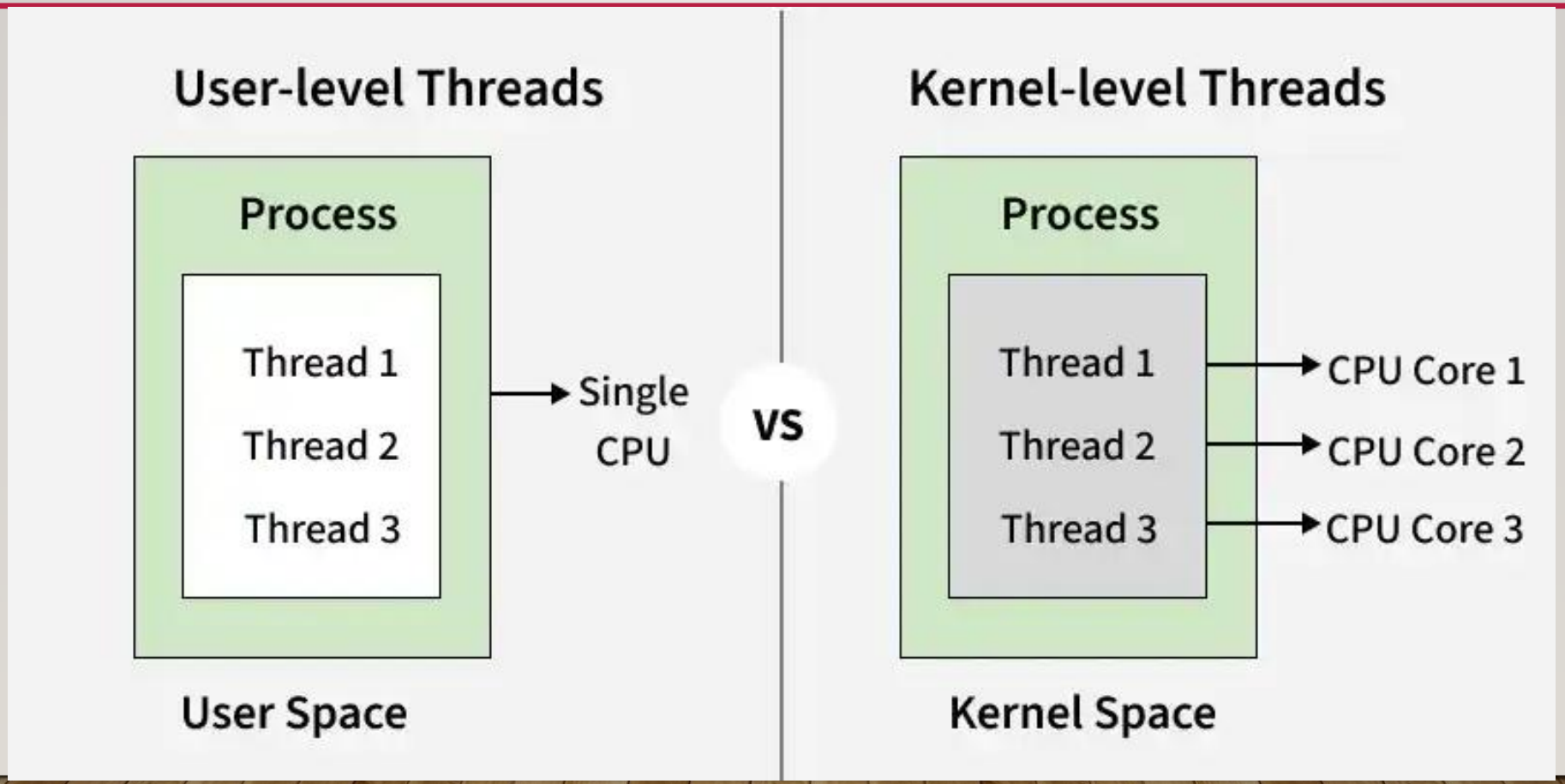
141 THREADS

- **Shared Resources**: Threads within the same process share code, data, and OS resources like open files.
- **Independent Execution**: Each thread has its own program counter (PC), register set, and stack space.
- **Lightweight**: Creating, terminating, and switching between threads is faster and less resource-intensive than creating full processes.
- **Responsiveness**: If one thread blocks (e.g., waiting for I/O), other threads in the process can continue executing.

142 THREADS

- Many software packages that run on modern desktop PCs are multithreaded.
- An application typically is implemented as a separate process with several thread of control.
- Types of Threads
  - **User-Level Threads (ULT)**: Managed entirely by application libraries without kernel intervention. They are fast but cannot take advantage of multi-core processors directly.
  - **Kernel-Level Threads (KLT):** Managed directly by the operating system. Supported by modern OS to provide true parallelism on multi-core systems.

143 THREADS


User-level Threads
Process
Thread 1
Thread 2
Thread 3
Single CPU
User Space
VS
Kernel-level Threads
Process
Thread 1
Thread 2
Thread 3
CPU Core 1
CPU Core 2
CPU Core 3
Kernel Space

144 THREADS

- User-Level Threads (ULTs)
  - Managed entirely in user space using a thread library; the kernel is unaware of them.
  - Switching between ULTs is fast since only program counter, registers, and stack need to be saved/restored.
  - Do not require system calls for creation or management, making them lightweight.
  - If one thread makes a blocking system call, the entire process (all threads) is blocked.

145 THREADS

- User-Level Threads (ULTs)
  - Scheduling is done by the application itself, which may not be as efficient as kernel-level scheduling.
  - Cannot fully utilize multiprocessor systems because the kernel schedules processes, not individual user-level threads.
- **Disadvantages**:
  - There is a lack of coordination between threads and operating system kernel.
  - User-level threads require non-blocking systems call.

THREADS

- Kernel-Level Threads (KLTs)
  - Managed directly by the operating system kernel; each thread has an entry in the kernel's thread table.
  - The kernel schedules each thread independently, allowing true parallel execution on multiple CPUs/cores.
  - Handles blocking system calls efficiently; if one thread blocks, the kernel can run another thread from the same process.
  - Provides better load balancing across processors since the kernel controls all threads.

147 THREADS

- Kernel-Level Threads (KLTs)
  - Context switching is slower compared to ULTs because it requires switching between user mode and kernel mode.
  - Implementation is more complex and requires frequent interaction with the kernel.
  - Large numbers of threads may add extra load on the kernel scheduler, potentially affecting performance.
- **Disadvantages**:
  - The kernel-level threads are slow and inefficient. For instance, threads operations are hundreds of times slower than that of user-level threads.

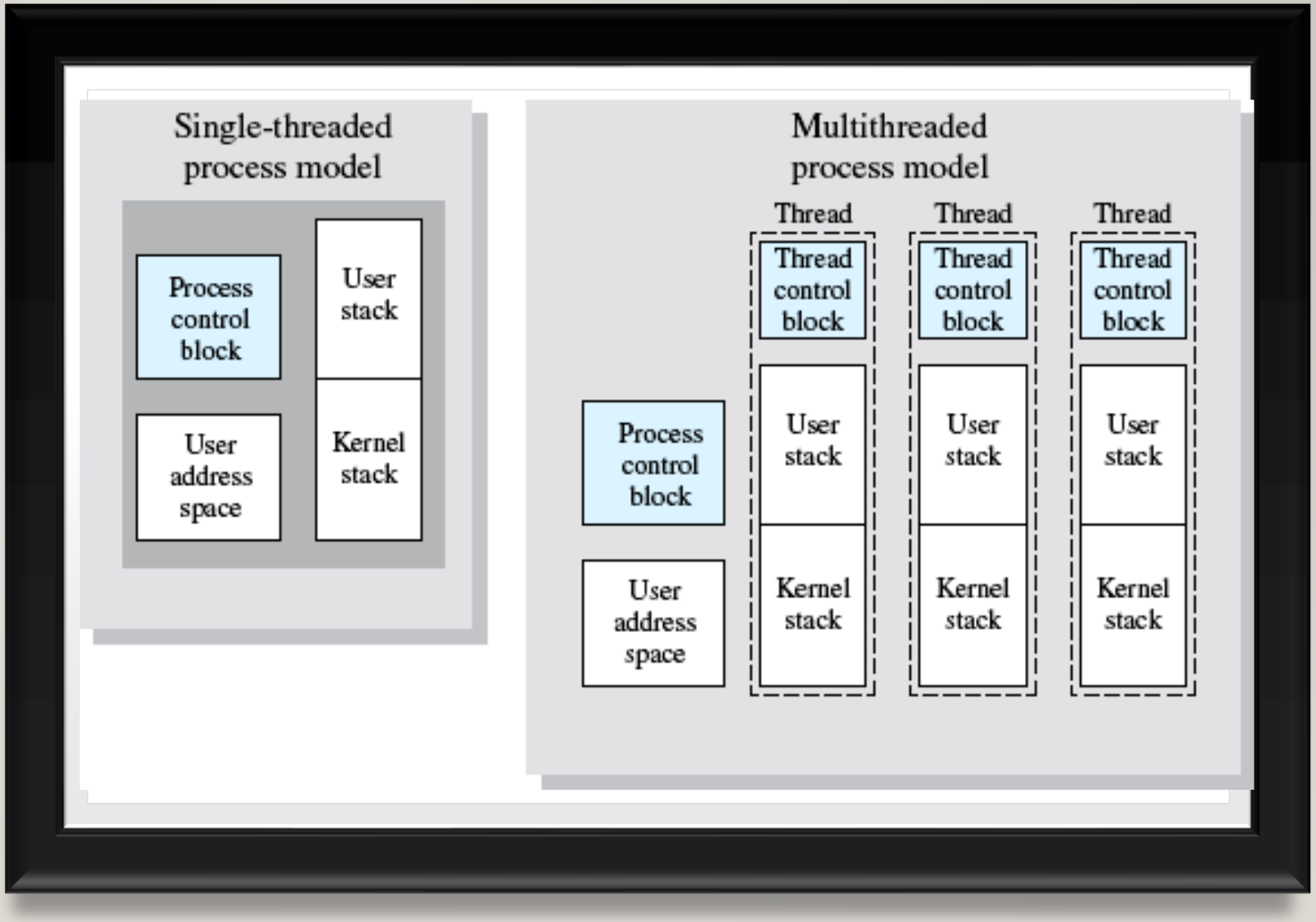
148 THREADS

- Why Threads?
  - **Improve Performance**: Threads allow multiple tasks to run at the same time (parallel or interleaved), making programs execute faster.
  - **Increase Responsiveness**: If one thread is busy, another can handle user actions, keeping the application responsive.
  - **Enable Concurrency**: Multiple operations like saving files, processing data, and handling user input can happen simultaneously.
  - **Better CPU Utilization**: On multi-core systems, threads can run on different cores, improving overall system performance.
  - **Efficient Resource Sharing**: Threads share the same memory and resources within a process, making communication faster and reducing overhead.

149 THREADS

- Components of Threads: These are the basic components of the Operating System.
  - **Stack Space**: Stores local variables, function calls, and return addresses specific to the thread.
  - **Register Set**: Hold temporary data and intermediate results for the thread's execution.
  - **Program Counter**: Tracks the current instruction being executed by the thread.

THREADS


Single-threaded process model
Process control block
User stack
User address space
Kernel stack
Multithreaded process model
Thread
Thread
Thread
Thread control block
Thread control block
Thread control block
Process control block
User stack
User stack
User stack
User address space
Kernel stack
Kernel stack
Kernel stack

151 THREADS

- Kernel must manage and schedule threads as well as processes.
- It require a full thread control block (TCB) for each thread to maintain information about threads.
- As a result, there is significant overhead and increased in kernel complexity.
- Many systems provide support for **both user and kernel threads**, resulting in different multithreading models.

152 THREADS: MODELS

- **One-to-one Model**: it maps each user thread to a kernel thread. It provides more concurrency by allowing another thread to run when a thread makes a blocking system call; it also allows multiple threads to run in parallel on multiprocessors. The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread. Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of threads supported by the system.

153 THREADS: MODELS

- **Many-to-One Model**: it maps many user-level threads to one kernel thread. Thread management is done in user space, so it is efficient, but the entire process will block if a thread makes a blocking system call. Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multiprocessors.

154 THREADS: MODELS

- **Many-to-Many Model:** it multiplexes many user-level threads to a smaller or equal number of kernel threads. The number of kernel threads may be specific to either a particular application or a particular machine . Whereas the many-to-one model allows the developer to create as many user threads as she wishes, true concurrency is not gained because the kernel can schedule only one thread at a time. The one-to-one model allows for greater concurrency, but the developer has to be careful not to create too many threads within an application.

THREADS: MODELS

Feature	Process	Threads
Memory	Separate (address)	Shared (address and files)
Time (creation)	Takes time	Take less time
Time (execution)	Execution and terminate	Less time to terminate
Execution (performance)	Slow	Very fast
Time (Switch)	More	Less
System call	Required	Not required
Resources (needed)	More to execute	Less to execute
Intercommunication	Bit difficult	Easy

INTER PROCESS COMMUNICATION

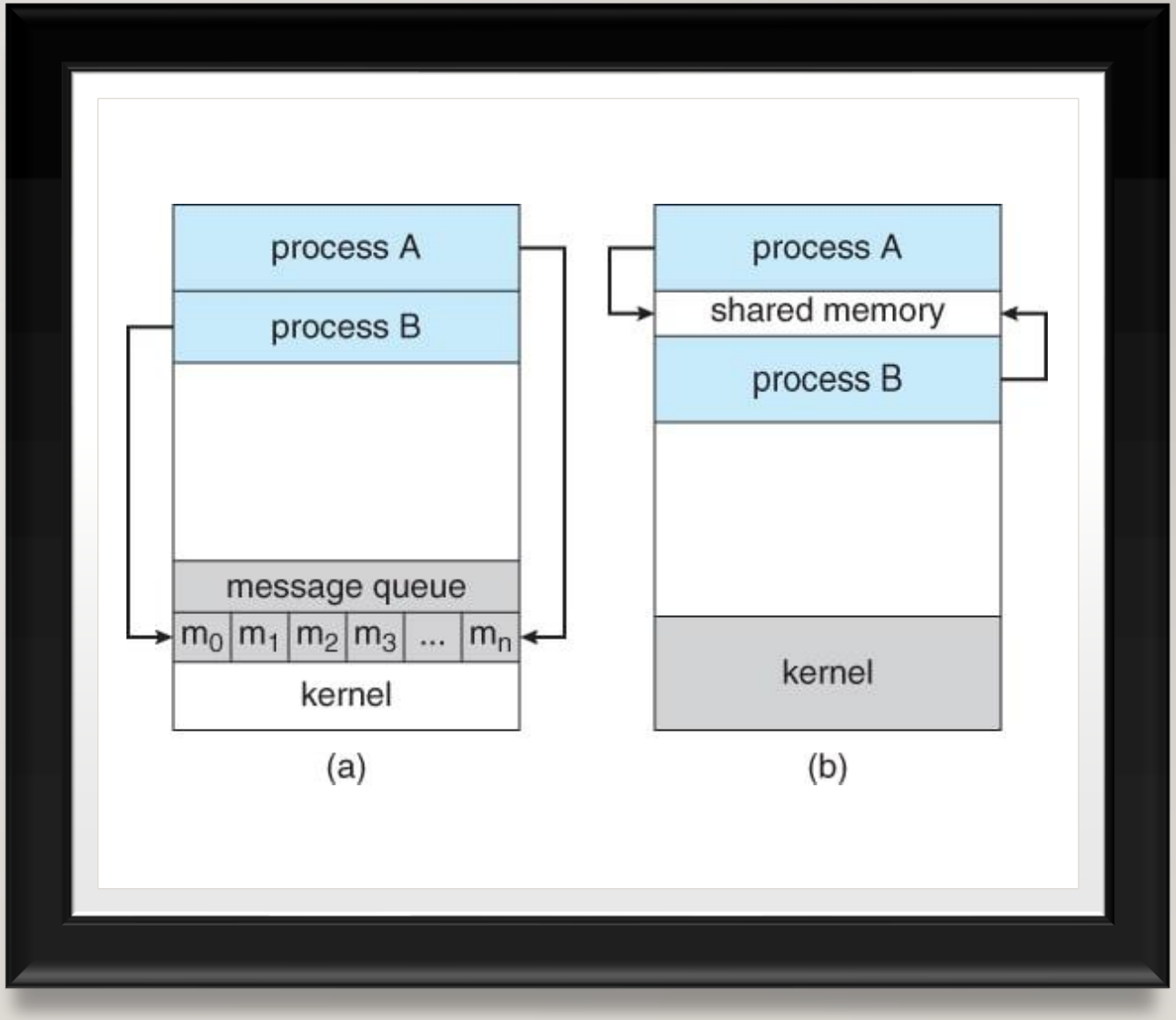
- Processes executing concurrently in an operating system can be either independent or cooperating.
  - **Independent Process**: A process that cannot affect or be affected by other processes. These processes do not share data with others.
  - **Cooperating Process**: A process that can be affected by or affect other processes. These processes share data and require communication to synchronize and exchange information.

INTER PROCESS COMMUNICATION

- Reasons for Process Cooperation:
  - **Information Sharing**: Multiple processes may need access to the same data (e.g., a shared file).
  - **Computation Speedup**: Large tasks can be broken into smaller sub-tasks to run in parallel, speeding up execution.
  - **Modularity**: A system can be constructed modularly by dividing functionality into separate processes or threads.
  - **Convenience**: Allows multiple tasks to run concurrently, improving system efficiency.

INTER PROCESS COMMUNICATION

- Cooperating processes require an **Inter-Process Communication (IPC)** mechanism to **exchange data** and **synchronize their operations**.
- The two primary IPC models are **Shared Memory** and **Message Passing**.



INTER PROCESS COMMUNICATION

- In shared-memory systems,
  - Cooperating processes communicate by sharing a region of memory.
  - Processes exchange information by reading and writing data to the shared memory region.
    - A process creates a **shared memory segment** in its address space.
    - Other processes that wish to communicate using this memory **must attach** the shared memory to their address space.
    - These processes can then exchange data by reading and writing in the shared region.
    - The OS does not control the data format or the memory location,
    - This responsibility lies with the processes.

INTER PROCESS COMMUNICATION

- Synchronization:
  - Processes must ensure that they do not write to the same memory location simultaneously.
  - One common problem in cooperative processes is the **Producer-Consumer** problem.
    - Producer Process: Produces data.
    - Consumer Process: Consumes data produced by the producer.
  - To allow both processes to run concurrently, they need a shared buffer (where the producer can place data, and the consumer can retrieve it)

INTER PROCESS COMMUNICATION

- Synchronization:
  - Unbounded Buffer: No limit on the buffer size.
    - The producer can continue producing items even if the consumer is slower in consuming.
    - The consumer may need to wait for new items, but the producer is not blocked.
  - Bounded Buffer: The buffer has a fixed size.
    - The producer must wait if the buffer is full, and the consumer must wait if the buffer is empty.
  - The shared buffer is often implemented as **a circular array** with two logical pointers:
    - **In Pointer**: Points to the next available position in the buffer.
    - **Out Pointer**: Points to the next item to be consumed.

INTER PROCESS COMMUNICATION

- Synchronization Mechanism:
  - To ensure the producer and consumer operate efficiently, synchronization mechanisms such as **mutexes** or **semaphores** are often used to avoid **race conditions** and ensure **mutual exclusion.**
  - To be covered later

INTER PROCESS COMMUNICATION

- Message Passing Systems
  - Message passing allows processes to communicate and synchronize without sharing the same address space, making it especially useful in distributed systems where processes may reside on different computers connected by a network.
- Message Passing Operations: They are at least two
  - send(message): Send a message to another process.
  - receive(message): Receive a message from another process.
- Messages can be fixed or variable in size:
  - Fixed-sized messages: Easier system implementation, but harder to program.
  - Variable-sized messages: More complex system-level implementation, but simpler for the programmer.

INTER PROCESS COMMUNICATION

- Message Passing Implementation Models
  - Direct or Indirect Communication
  - Synchronous or Asynchronous Communication
  - Automatic or Explicit Buffering

INTER PROCESS COMMUNICATION: DIRECT COM

- It is a message-passing method where processes explicitly name the sender or receiver to communicate
  - send (P, message) – send a message to process P
  - receive(Q, message) – receive a message from process Q Properties of communication link
- Links are established automatically.
- A link is associated with exactly one pair of communicating processes.
- Between each pair there exists exactly one link.
- The link may be unidirectional but is usually bi-directional.

INTER PROCESS COMMUNICATION: DIRECT COM

- Properties:
  - Explicit Naming: The sender and receiver must know each other (symmetric addressing), or the receiver does not need to know the specific sender (asymmetric addressing).
  - A link is established automatically between processes and nowadays a bidirectional link.
  - Tight Coupling, because processes must know each other's IDs, changing a process's name requires changing all direct references

INTER PROCESS COMMUNICATION: DIRECT COM

- Link Capacity
  - Zero Capacity: no messages can wait in the queue; the sender must block until the receiver receives the message.
  - Bounded Capacity: a finite queue of messages; the sender blocks only if the queue is full.
  - Unbounded Capacity: An infinite queue; the sender never waits.

INTER PROCESS COMMUNICATION: DIRECT COM

- Advantage: Simpler and more direct interaction between specific, known processes.
- Disadvantage: Harder to use in modular programming because of the need to explicitly name processes, leading to tighter coupling

INTER PROCESS COMMUNICATION: INDIRECT COM

- It is a message-passing mechanism where processes interact via shared intermediate structures like mailboxes or ports rather than direct addressing.
- A sender deposits a message in a mailbox, and a receiver retrieves it, providing flexibility and decoupling in communication.
- Each mailbox has a unique **ID**
- Processes communicate through shared mailboxes.
- Mailboxes are typically managed by the operating system, allowing messages to be stored until retrieved.

INTER PROCESS COMMUNICATION: INDIRECT COM

- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes.
  - Each pair of processes may share several communication links.
  - Link may be unidirectional or bi-directional.

INTER PROCESS COMMUNICATION: INDIRECT COM

- Operations
  - create a new mailbox
  - send and receive messages through mailbox
  - destroy a mailbox
- Primitives are defined as:
  - send(A, message) – send a message to mailbox A
  - receive(A, message) – receive a message from mailbox A

INTER PROCESS COMMUNICATION: INDIRECT COM

- Characteristics:
  - Decoupling, the sender and receiver do not need to know each other's identifiers, only the ID of the shared mailbox.
  - Flexibility, flexible and scalable communication, as multiple processes of different size can share a single mailbox.
- There are two types of mailbox ownership:
  - Process-owned mailbox: A process can only receive messages through its mailbox, and it disappears when the process terminates.
  - OS-owned mailbox: A mailbox exists independently of any process, and the OS manages its creation, deletion, and message passing.

INTER PROCESS COMMUNICATION: INDIRECT COM

- Advantages
  - Decoupling: Processes are not directly dependent on each other's state.
  - Flexibility: Multiple sender and receiver configurations are possible
- Disadvantages
  - Overhead: Managing mailboxes requires system resources and overhead to maintain synchronization.
  - Complexity: Can be more complex to implement compared to direct communication.

INTER PROCESS COMMUNICATION: SYNCHRONIZATION

- Synchronization determines how send and receive operations block or proceed.
  - **Blocking Send**: The sender is blocked until the message is received or placed in the mailbox.
  - **Non-blocking Send**: The sender sends the message and continues execution without waiting.
  - **Blocking Receive**: The receiver is blocked until a message is available.
  - **Non-blocking Receive**: The .receiver retrieves either a valid message or a null.

INTER PROCESS COMMUNICATION: SUMMARY

- A process is a program in execution and changes state during its lifetime.
- The states include:
  - New: The process is being created.
  - Ready: The process is ready to execute, waiting for the CPU.
  - Running: The process is currently executing.
  - Waiting: The process is waiting for an event (like I/O completion).
  - Terminated: The process has finished execution.

INTER PROCESS COMMUNICATION: SUMMARY

- Process Control Block (PCB):
  - Each process is represented by its own Process Control Block (PCB) in the OS.
  - A process that is not executing is placed in a waiting queue, which can be either an I/O request queue or a ready queue.
  - The ready queue holds processes that are ready to execute and waiting for the CPU.

INTER PROCESS COMMUNICATION: SUMMARY

- Scheduling:
  - Long-term (Job) Scheduling: Selects processes to contend for the CPU, influenced by resource allocation, especially memory management.
  - Short-term (CPU) Scheduling: Selects one process from the ready queue to execute.
- Types of Processes:
  - Independent Processes: These processes do not affect or share data with other processes.
  - Cooperating Processes: These processes interact with each other and require an Interprocess Communication (IPC) mechanism.

INTER PROCESS COMMUNICATION: SUMMARY

- Parent processes can create child processes.
  - The parent can either wait for the child to terminate before proceeding or allow them to execute concurrently.
- Reasons for concurrent execution include:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience

INTER PROCESS COMMUNICATION: SUMMARY

- A **thread** is a single sequence of instructions within a process.
  - Each thread within a process shares the same memory space, but they can execute independently.
  - A multithreaded process contains multiple threads running concurrently within the same process.
  - This allows for parallelism, enabling efficient use of CPU resources.

INTER PROCESS COMMUNICATION: SUMMARY

- Types of Threads:
  - User-Level Threads: These threads are created and managed by the user-level thread library (Pthreads), not by the operating system. The kernel is unaware of their existence, meaning it cannot schedule or manage them directly.
  - Kernel-Level Threads: These threads are created and managed by the operating system kernel. The kernel schedules these threads and is aware of their existence. If one kernel thread is blocked, the operating system can schedule another thread from the same process.

INTER PROCESS COMMUNICATION: SUMMARY

- Thread Models:
  - **Many-to-One Model**: In this model, multiple user-level threads are mapped to a single kernel thread. This model allows multiple threads within a process to run concurrently from the user's perspective, but the operating system can only schedule one kernel thread at a time.
  - **One-to-One Model**: In this model, each user-level thread corresponds to a kernel thread. This allows each thread to be scheduled independently by the operating system.
  - **Many-to-Many Model**: This model multiplexes many user threads onto a smaller or equal number of kernel threads. This allows a process to use multiple kernel threads while still managing many user threads.

INTER PROCESS COMMUNICATION: SUMMARY

- Inter-process Communication (IPC):
  - **Shared Memory**: Processes communicate by reading/writing to shared memory regions. The OS provides the memory, but application programmers manage the communication.
  - **Message Passing**: Processes exchange messages. The OS typically provides the communication mechanism.
- Both schemes (shared memory and message passing) can be used simultaneously in an OS.

PROCESS SYNCHRONIZATION

- Process synchronization refers to various mechanisms used to ensure the orderly execution of **cooperating processes** that **share** a **logical address space,** ensuring data consistency is maintained.
- **Concurrent execution** occurs when processes share CPU time and are executed in overlapping time periods.
- **Parallel execution**, on the other hand, involves two or more processes executing simultaneously on **separate processing cores**.

PROCESS SYNCHRONIZATION

- **Context of Process Execution**:
  - When processes execute **concurrently or in parallel**, one process may only partially complete execution before another process is scheduled.
  - In **concurrent execution**, a process can be interrupted at any point in its instruction stream, and another process might begin execution immediately, potentially **causing issues when both processes manipulate shared data**.
  - **Parallel execution** involves multiple processes running at the same time on different processors or cores, which may lead to **similar data integrity issues** when accessing shared data.

PROCESS SYNCHRONIZATION

- Problems Arising from Concurrent Execution is Race Condition.
  - A race condition occurs when multiple processes or threads **concurrently** access and manipulate **shared data**, and the outcome depends on the order or timing of the execution of these processes.
- Example: two processes simultaneously attempt to increment a shared counter variable, the outcome can vary based on the order in which the increments occur, leading to inconsistent or incorrect results.

PROCESS SYNCHRONIZATION

- Improper Synchronization in Inter Process Communication Environment leads to following problems:
  - **Inconsistency**: when two or more processes access shared data at the same time without proper synchronization. Then conflicting changes, where one process's update is overwritten by another, data become unreliable and incorrect.
  - **Loss of Data:** occurs when multiple processes try to write or modify the same shared resource without coordination. Leading to incomplete or corrupted data.
  - **Deadlock**: which means that two or more processes get stuck, each waiting for the other to release a resource. None of the processes can continue, the system becomes unresponsive and none of the processes can complete their tasks.

PROCESS SYNCHRONIZATION

- Role of Synchronization in IPC
  - **Preventing Race Conditions**: Ensures processes don't access shared data at the same time, avoiding inconsistent results.
  - **Mutual Exclusion**: Allows only one process in the critical section at a time.
  - **Process Coordination**: Lets processes wait for specific conditions.
  - **Deadlock Prevention**: Avoids circular waits and indefinite blocking by using proper resource handling.
  - **Safe Communication**: Ensures data/messages between processes are sent, received and processed in order.
  - **Fairness**: Prevents starvation by giving all processes fair access to resources.

PROCESS SYNCHRONIZATION

- Types of Process Synchronization
  - **Competitive**: Two or more processes are said to be in Competitive Synchronization if and only if they compete for the accessibility of a shared resource.
    - Lack of Proper Synchronization among Competing process may lead to either Inconsistency or Data loss.
  - **Cooperative**: Two or more processes are said to be in Cooperative Synchronization if and only if they get affected by each other i.e. execution of one process affects the other process.
    - Lack of Proper Synchronization among Cooperating process may lead to Deadlock.

PROCESS SYNCHRONIZATION

- Required conditions for Process Synchronization
  - **Critical Section**:
    - A critical section is a code segment that can be accessed by only one process at a time.
    - The critical section contains shared variables that need to be synchronized to maintain the consistency of data variables.
    - The critical section problem means designing a way for cooperative processes to access shared resources without creating data inconsistencies.

PROCESS SYNCHRONIZATION

- Required conditions for Process Synchronization
  - **Race Condition**:
    - A race condition is a situation that may occur inside a critical section.
    - This happens when the result of multiple process/thread execution in the critical section differs according to the order in which the threads execute.

PROCESS SYNCHRONIZATION

- Required conditions for Process Synchronization
  - **Pre-emption**:
    - Preemption is when the operating system stops a running process to give the CPU to another process.
    - This allows the system to make sure that important tasks get enough CPU time.
    - This is important as mainly issues arise when a process has not finished its job on shared resource and got preempted.
    - The other process might end up reading an inconsistent value if process synchronization is not done.

PROCESS SYNCHRONIZATION: CRITICAL SECTION

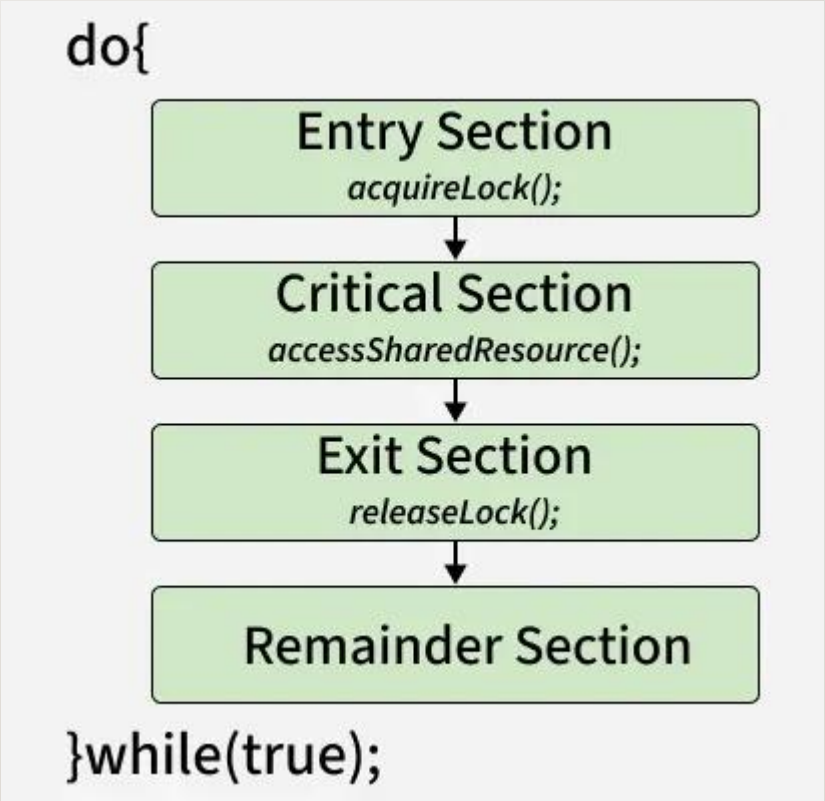
- A critical section is a part of a program where shared resources (memory, files, variables) are accessed by multiple processes or threads.
- To avoid problems such as race conditions and data inconsistency, only one process/thread should **execute** the critical section at a time using synchronization techniques.
- This ensures that operations on shared resources are performed safely and predictably.

PROCESS SYNCHRONIZATION: CRITICAL SECTION

- Structure of a Critical Section
  1. **Entry** Section
     - The process requests permission to enter the critical section.
     - Synchronization tools (mutex, semaphore) are used to control access.
  2. **Critical** Section: The actual code where shared resources are accessed or modified.
  3. **Exit** Section: The process releases the lock or semaphore, allowing other processes to enter the critical section.
  4. **Remainder** Section: The rest of the program that does not involve shared resource access.

PROCESS SYNCHRONIZATION: CRITICAL SECTION

- Structure of a Critical Section



PROCESS SYNCHRONIZATION: CRITICAL SECTION

- A good critical section solution must ensure:
  - **Correctness** - Shared data should remain consistent.
  - **Efficiency** - Should minimize waiting and maximize CPU utilization.
  - **Fairness** - No process should be unfairly delayed or starved.
- Requirements of Critical Section Solutions
  - Mutual Exclusion
    - At most one process can be inside the critical section at a time.
    - Prevents conflicts by ensuring no two processes update the shared resource simultaneously.

PROCESS SYNCHRONIZATION: CRITICAL SECTION

- Requirements of Critical Section Solutions
  - Progress
    - If no process is in the critical section, and some processes want to enter, the choice of who enters next should not be postponed indefinitely.
    - Ensures that the system continues to make progress rather than getting stuck.
  - Bounded Waiting
    - There must be a limit on how long a process waits before it gets a chance to enter the critical section.
    - Prevents starvation, where one process is repeatedly bypassed while others get to execute.

PROCESS SYNCHRONIZATION: CRITICAL SECTION

- Examples of critical section
  - Banking System (ATM / Online Banking)
    - Critical Section: Updating an account balance during a deposit or withdrawal.
    - Issue if not handled: Two simultaneous withdrawals could result in an incorrect final balance due to race conditions.
  - Ticket Booking System (Airlines, Movies, Trains)
    - Critical Section: Reserving the last available seat.
    - Issue if not handled: Two users may be shown the same available seat and both may book it, leading to overbooking.